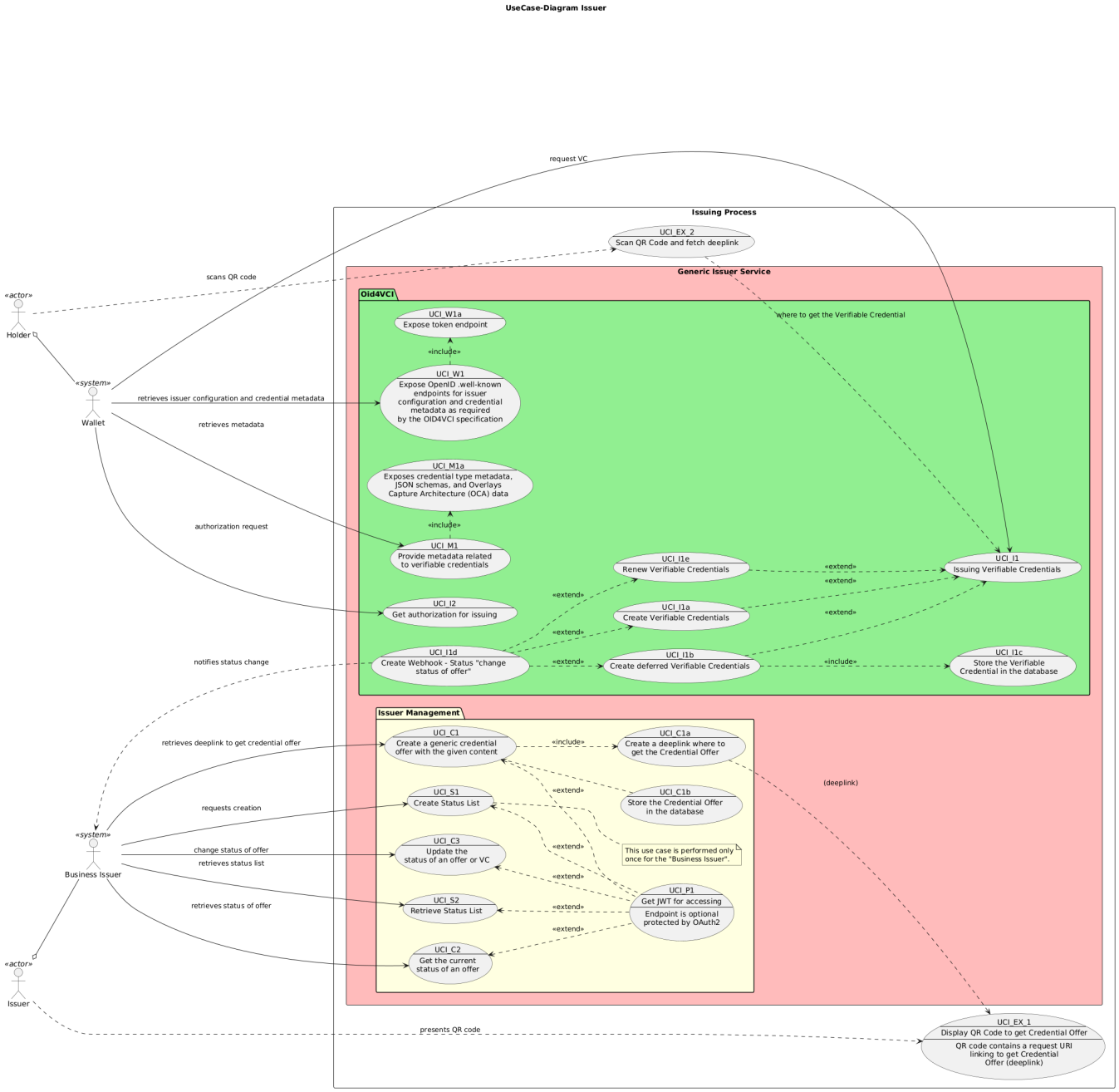
Issuer Documentation (pdf export)

- 1. Business Context
  - 1.1. Use Cases
  - 1.2. Actors
  - 1.3. External Systems
  - 1.4. Interactions
- 2. Technical Context
  - 2.1. Core System
  - 2.2. Context
- 3. Building Block View
  - 3.1. Building Blocks - Top Level
  - 3.2. Building Blocks - Level 1
  - 3.3. Issuer Usage
- 4. Detailed Class View
  - 4.1. Web Layer (Controllers)
- 5. Interfaces
- 6. Interface - Credential Management API Specification (IF-114)
  - 6.1. Overview
  - 6.2. Data Model Summary
  - 6.3. Endpoints
  - 6.4. Error Handling
  - 6.5. Field & Enum Reference
  - 6.6. Examples
- 7. Interface - Status List Management API Specification (IF-113)
- 8. Interface - Well-known Endpoints API Specification (IF-112)
  - 8.1. Overview
  - 8.2. Headers
  - 8.3. Data Models
  - 8.4. Endpoints
  - 8.5. Content Negotiation
  - 8.6. Default Exception Handling
- 9. Interface - Issuance Controller API Specification (IF-111)
  - 9.1. Headers
  - 9.2. Data Models
  - 9.3. Endpoints
  - 9.4. Default Exception Handling
- 10. Interface - Credential Metadata API Specification (IF-110)
  - 10.1. Overview
  - 10.2. Headers
  - 10.3. Endpoints
  - 10.4. Content Types and Negotiation
  - 10.5. Default Exception Handling
- 11. Interface - Issuer Renewal API Specification (IF-201)
  - 11.1. Data Models
  - 11.2. Endpoints
  - 11.3. Example
- 12. Interface - Webhook Callback API Specification (IF-202)
  - 12.1. Data Models
  - 12.2. Functionality
  - 12.3. Configuration
  - 12.4. Example
- 13. Runtime - Issuance Process - Complete
  - 13.1. Overview of oid4vci flow - Pre-Authorized Code Flow
  - 13.2. Detailed View
  - 13.3. Deferred Mode
- 14. Runtime - Initializing a new Status List
- 15. Runtime - Update Status List
- 16. Runtime - Renewal Flow - Batch Issuance
  - 16.1. Abstract
  - 16.2. Core Objectives and Methodology
  - 16.3. High-Level Technical Flow (OID4VCI and OAuth 2.0)
  - 16.4. Fundamentals and Architectural Components
  - 16.5. Technical Standards and Protocols
  - 16.6. Technical Implementation
  - 16.7. Credential Management Status
  - 16.8. Credential Offer Status
- 17. Runtime - Issuer - State-Event-Diagramm
  - 17.1. Credential Management Status
  - 17.2. Credential Offer Status
- 18. Runtime - Trust Infrastructure Interactions
- 19. Runtime - Issuance with Transaction Codes
  - 19.1. Context and Scope
  - 19.2. Runtime View
- 20. Crosscutting - Branches / Releases Concept

- 20.1. Naming Conventions of Branches
- 20.2. Semantic Versioning (SemVer) Overview
- 20.3. Releases

1. Business Context

1.1. Use Cases



UC Number	Name	Description	Link
UCI_EX_1	Display QR Code to get Credential Offer	The Issuer displays a QR code containing a deeplink (request URI) that allows the Holder to initiate the credential offer retrieval process.	
UCI_EX_2	Scan QR Code and fetch deeplink	The Holder scans the QR code and uses the deeplink to fetch the credential offer from the issuer service.	

UCI _S1	Create Status List	Allows the Business Issuer to initialize a new status list resource, which will be used to track the revocation or validity status of verifiable credentials issued by the system. This operation is typically performed once per issuer or credential type to enable status management for all subsequently issued credentials.	
UCI _S2	Retrieve Status List	Enables the Business Issuer to fetch the current status list resource, which contains the up-to-date status (such as valid, revoked, or suspended) of all verifiable credentials managed by the issuer. This operation is essential for monitoring, auditing, and verifying the validity of issued credentials.	
UCI _C1	Create a generic credential offer	Enables the Business Issuer to generate a new credential offer with custom content and properties. This offer serves as the basis for issuing verifiable credentials to Holders and includes all necessary information for the credential issuance process. The created offer can be retrieved and used by authorized parties to initiate credential issuance.	
UCI _C1a	Create a deeplink for Credential Offer	Generates a unique deeplink (URL) that points to the created credential offer, enabling Holders or Wallets to retrieve the offer directly. This deeplink encapsulates all necessary information for the credential retrieval process and is embedded in a QR code.	
UCI _C1b	Store the Credential Offer in the database	Persists the created credential offer in the database for later retrieval and issuance.	
UCI _C2	Get the current status of an offer	Allows the Business Issuer to retrieve the current status (such as DEFERRED, READY, ISSUED or REVOKED) of a specific credential offer.	
UCI _C3	Update the status of an offer or VC	Allows the Business Issuer to change the status (such as suspended, revoked, or reactivated) of an existing verifiable credential. This operation is essential for managing the credential lifecycle, ensuring compliance, and supporting scenarios like credential revocation or reinstatement.	
UCI _W1	Expose OpenID . well-known endpoints	Provides endpoints as required by the OID4VCI specification, allowing Wallets to retrieve issuer configuration and credential metadata. This enables interoperability and discovery of issuer capabilities, supported credential types, and endpoints necessary for the verifiable credential issuance process.	
UCI _W 1a	Expose token endpoint	Exposes the OAuth2 token endpoint for credential issuance authorization.	
UCI _M1	Provide metadata related to VCs	Supplies metadata about supported verifiable credentials, such as types, schemas, and overlays.	
UCI _M1a	Expose credential type metadata	Exposes detailed metadata, JSON schemas, and OCA data for credential types.	
UCI _I1	Issuing Verifiable Credentials	Handles the process of issuing verifiable credentials to Holders, including validation and packaging.	
UCI _I1a	Create Verifiable Credentials	Issues a verifiable credential immediately upon request, using the provided bearer token and credential properties.	
UCI _I1b	Create deferred Verifiable Credentials	Issues a verifiable credential in a deferred manner, requiring a transaction ID and access token.	
UCI _I1c	Store the Verifiable Credential	Persists the issued verifiable credential in the database for future reference and status tracking.	
UCI _I1d	Create Webhook - Status change	Notifies the Business Issuer via webhook when the status of an offer or credential changes (e.g., issued, revoked).	
UCI _I1e	Renew Verifiable Credentials	Renews a verifiable credential, requiring an access token. (Creating a new credential offer but linked to the same management entity as the previous verifiable credential.	
UCI _I2	Get authorization for issuing	Allows the Wallet to obtain authorization (OAuth2 bearer token) for credential issuance using a pre-authorized code.	
UCI _P1	Get JWT for accessing	Management Endpoint is optional protected by OAuth2	

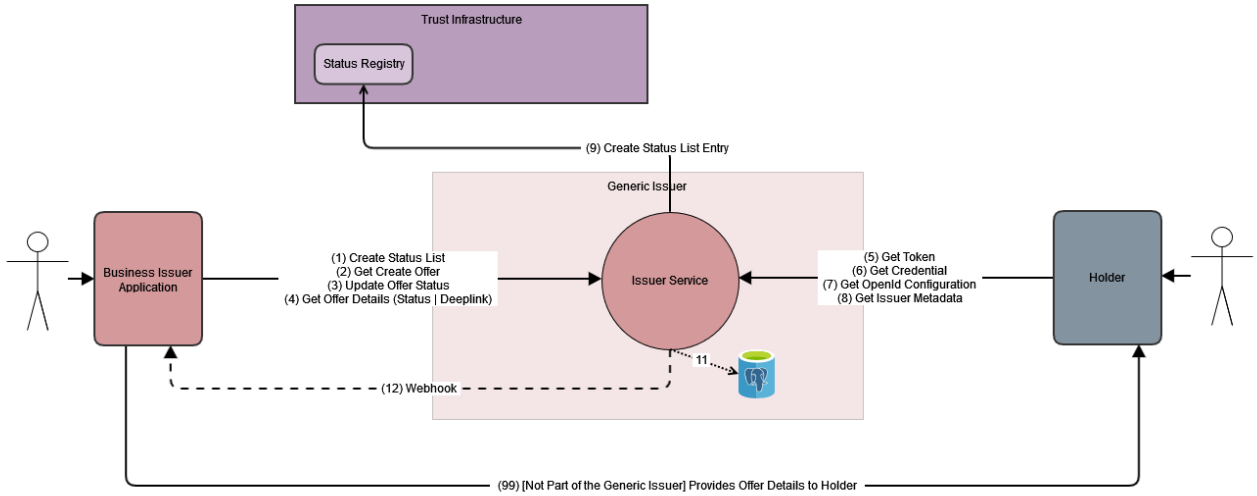
1.2. Actors

Actor	Description
Business Issuer Application	Application that initializes the process and requests the process result. The service can also initialize status list and update the status for a credential via Issuer Agent Management.
Holder	Mobile Wallet for iOS and Android which holds and builds the verifiable presentations.

1.3. External Systems

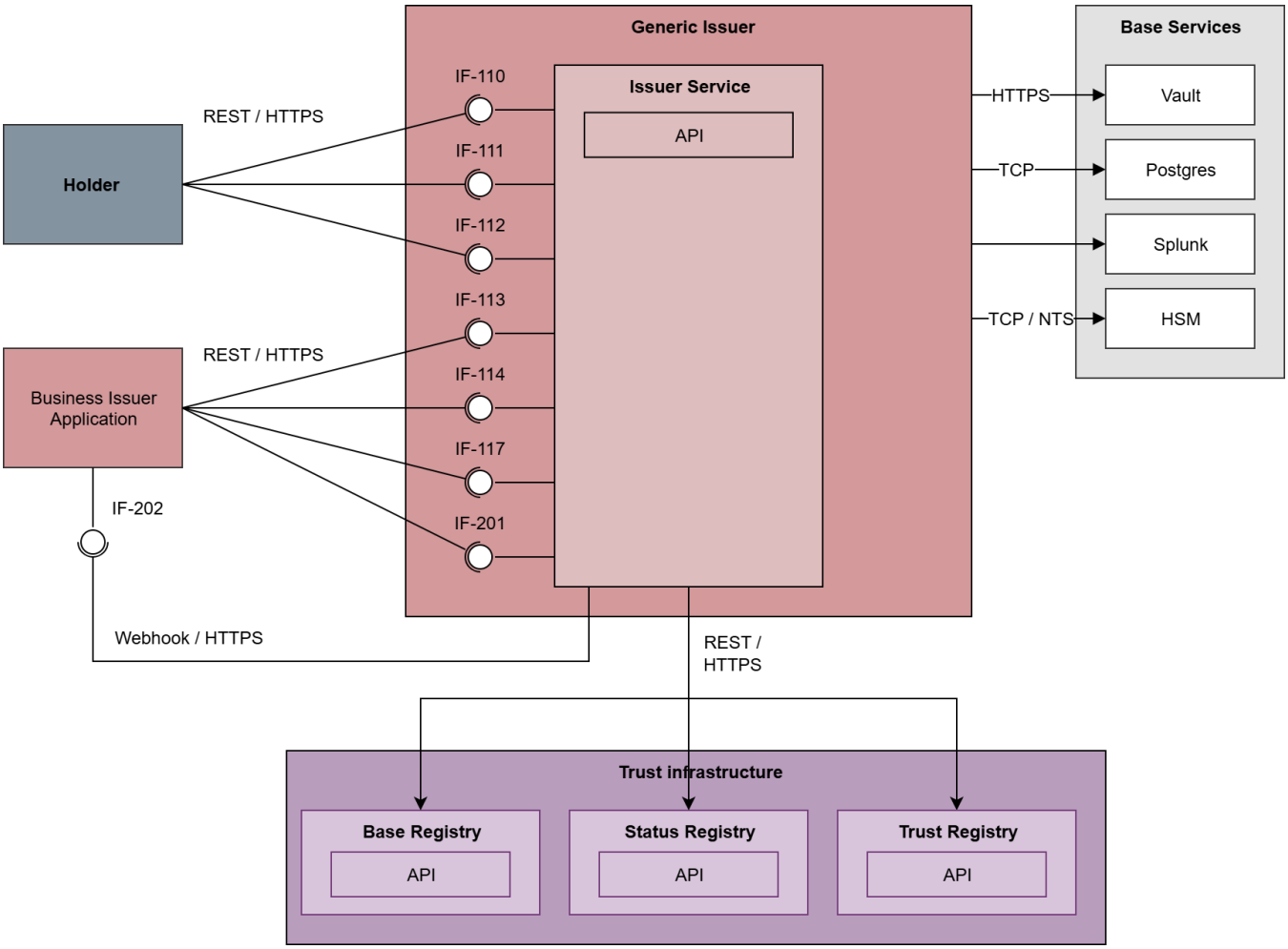
System	Description
Status Registry	Is part of the Trust Infrastructure. Stores the Status Lists created by the Issuer Agent Managment Service and used for example by the Generic Verifier OID4VP or the Holder to get Status Informations.

1.4. Interactions



Number	Interaction Origin	Interaction Target	Description
1	Business Issuer	Issuer Service (Management)	Business Issuer creates a status list on the Issuer Service with an API call. The Status List must be created before by the Business Issuer on the Status Registry. This interaction helps the Issuer Service to store the metadata about the Status List such as which is the next free entry in the status list.
2	Business Issuer	Issuer Service (Management)	The Business Issuer creates an Offer on the Issuer Service which is then stored on the db (10) and the status list is updated (9).
3	Business Issuer	Issuer Service (Management)	The Business Issuer can update the Status of a Credential on the Issuer Service, which updates the Status Registry (9).
4	Business Issuer	Issuer Service (Management)	The Business Issue can request the details of an offer including Status of the Process and Deeplink.
5	Holder	Issuer Service (OID4VCI)	The Holder can get the token with information provided by the deeplink (grantType and pre-auth-code), which is sent from the Business Issuer Application.
6	Holder	Issuer Service (OID4VCI)	The holder can get the credential from the Issuer service with the token from (5).
7	Holder	Issuer Service (OID4VCI)	With the OpenId-Configuration Endpoint the Holder can get the infromation needed to get the authentication token (5) such as token endpoint.
8	Holder	Issuer Service (OID4VCI)	Holder can get the issuer metadata. Which defines the different credentials provided by the issuer.
9	Issuer Agent Management	Trust Infrastructure - Status Registry	Issuer Service can initialize and update status lists on the Status Registry.
10	Issuer Agent Management	Postgres - Database	Issuer Service stores the offer data on the database and reads the data if requested by the business issuer.
11	Issuer Agent OID4VCI	Postgres - Database	Issuer Service reads the offer from the database to issue the data to the holder. The Issuer Service also updates the status of the offer.
12	Issuer Agent	Business Issuer	Optional Webhook. Triggered on events such as when a VC is issued or an issuing related error occurs like the key attestation service being not trusted.

2. Technical Context



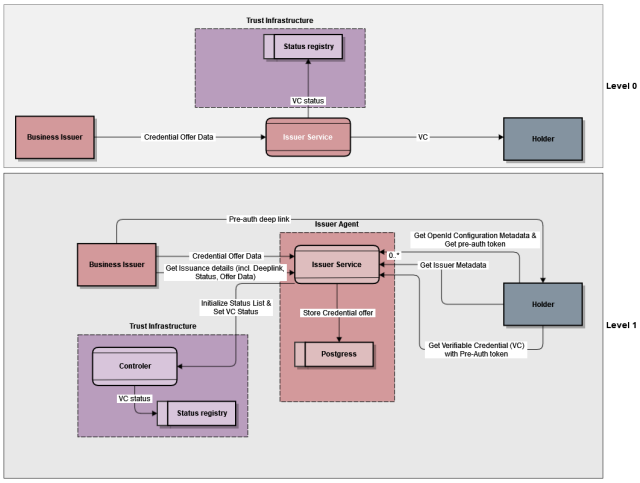
2.1. Core System

System	Description
Issuer Service	Component responsible for initiating and managing the credential issuance process. It offers an API for the Business Issuer Application and interfaces with the Trust Infrastructure Status Registry to manage credential status. The component includes endpoints for authentication, credential retrieval, and metadata provisioning.

2.2. Context

System	Description
Postgres	Database to store the business objects.
Splunk	Used for application logging and audit logs.
Vault	Secure storage of credentials.

3. Building Block View



Actors

Actor	Scope	Actions
Business Issuer Application	Internal	• Creates Credential offers • Reads Credential Offer and Verifiable Credential Status • Sets Verifiable Credential Status • Initializes new Status Lists
Holder	Public	• Receives Offer Deeplink from Business Issuer • Fetches OpenId Config Metadata in order to get the Auth Token • Gets the Oauth Token with the information retrieved from the deeplink • Checks the Issuer metadata to get additional information what type of verifiable credentials can be offered. • Fetches the Verifiable Credential in the desired format.

3.1. Building Blocks - Top Level

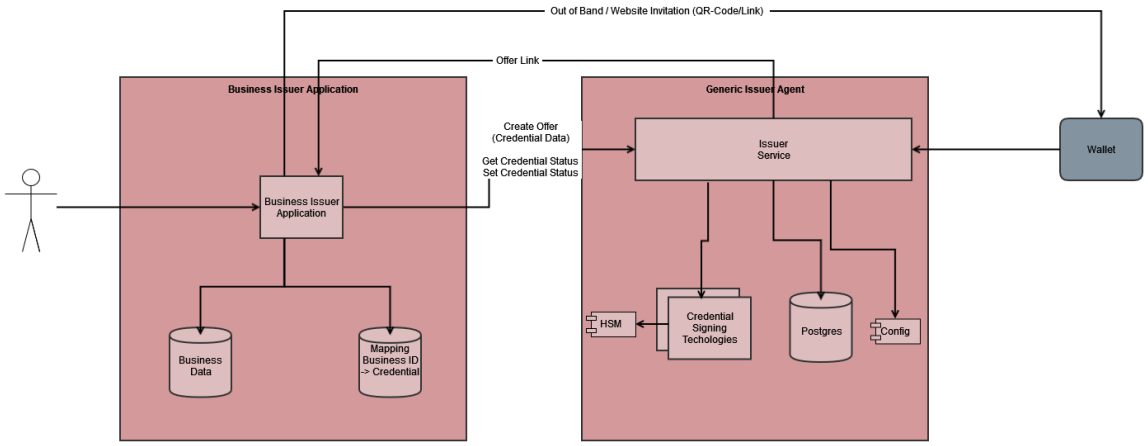
Component	Responsability
Generic Issuer Agent	The generic issuer contains the means to create verifiable credentials. It is also responsible to set and update the status of a credential.
Business Issuer	The business issuer starts the offering process by using the generic issuers create offer api. The Business issuer is responsible to hold the data created by the offering process (offered data, status list, status list position). It also provides the offer deeplink to the holder appication to start the redeeming process.
Holder	The interaction between holder is initialized by the Business issuer, which sends the Offer-deeplink to the holder, which contains information about the Issuer and how to retrieve the credential. The holder is responsible to get the verifiable credentials by using the OpenId-Metadata endpoint, Token endpoint and the credential endpoint. The holder is also responsible to store the vc.
Trust Infrastructure	The trust infrastructure is responsible for createing, storing and retrieving data necessary for the issuance process. The data is publically available and can only be updated by the owner.

3.2. Building Blocks - Level 1

Component	Part of Component	Responsability
Issuer Agent Management	Generic Issuer Agent	Is responsible for the internal communication with the Business Issuer. It provides an API to: • Create credential offers • Initialize Status list (status list entry must already be registered in the Status Registry) • Update Status of a verifiable credential (VC) • Provide information about the offer and the status of the process It provides the offer data to the Issuer Agent OID4VCI via Database.

Issuer Agent OID4VCI	Generic Issuer Agent	Is responsible for the public communication with the Holder. It provides an API to: • Get OpenID-Metadata, which defines what is required use the token endpoint • The issuer-metadata endpoint, which provides information about the different credentials • The Token endpoint used to authenticate and then get the credential • The credential endpoint, which provides the actual credential The component is also responsible to update the status of the credential once it is issued and write the status to the Database
Database (Postgres)	Generic Issuer Agent	Database to store the offer details and the status list information. This is the only communication channel between the Issuer Management and the Issuer OID4VCI component.
Controller	Trust Infrastructure	The controller of the Trust Registry is the Service used to handle the communication between the Issuer Agent Management service and the registries.
Status Registry	Trust Infrastructure	Status Registry is used to store the status list and make them publicly available.

3.3. Issuer Usage



4. Detailed Class View

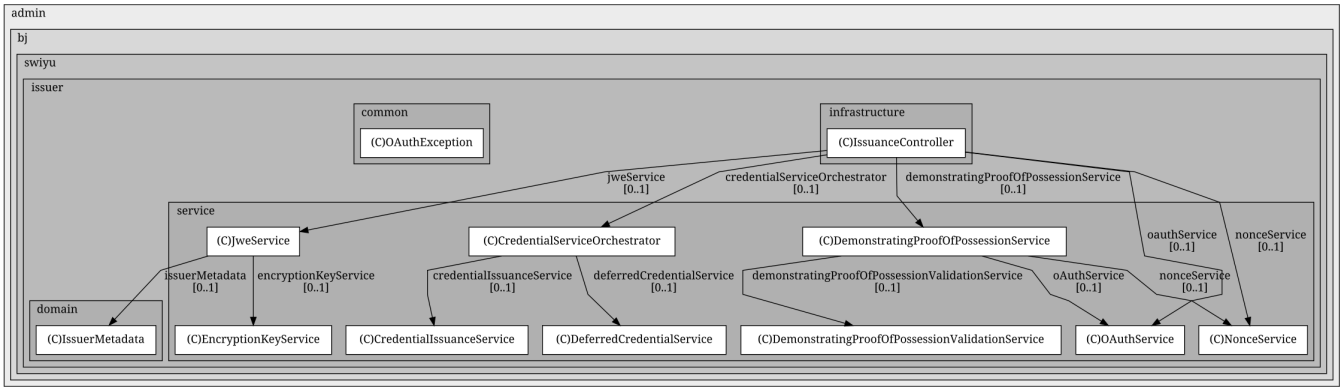
4.1. Web Layer (Controllers)

4.1.1. Signer

4.1.1.1. IssuanceController (infrastructure.web.signer)

Entry point for credential issuance to wallets/clients.

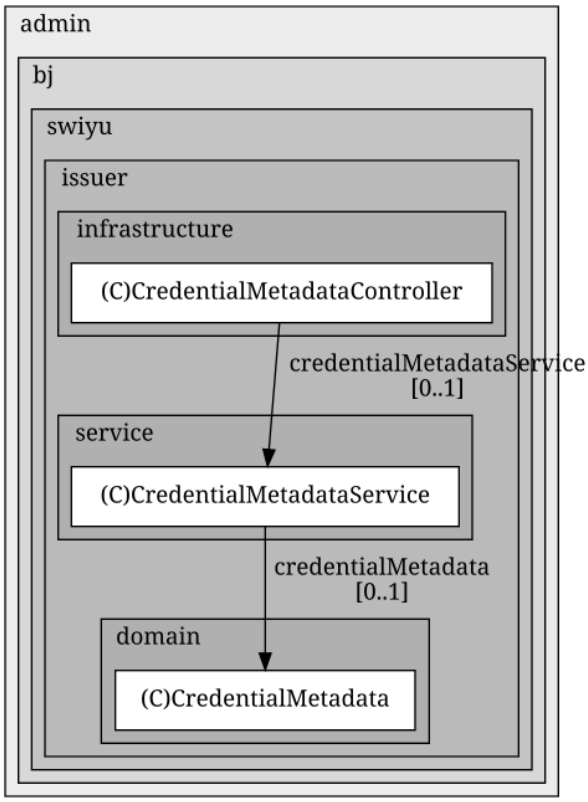
- Delegates issuance flow to CredentialServiceOrchestrator.
- Uses DemonstratingProofOfPossessionService for DPoP / proof-of-possession handling.
- Uses JweService for encrypting responses.
- Uses NonceService and OAuthService for nonce and OAuth/token-related checks.
- Reacts to OAuth-specific errors via OAuthException.



4.1.1.2. CredentialMetadataController (infrastructure.web.signer)

Exposes credential metadata for wallets.

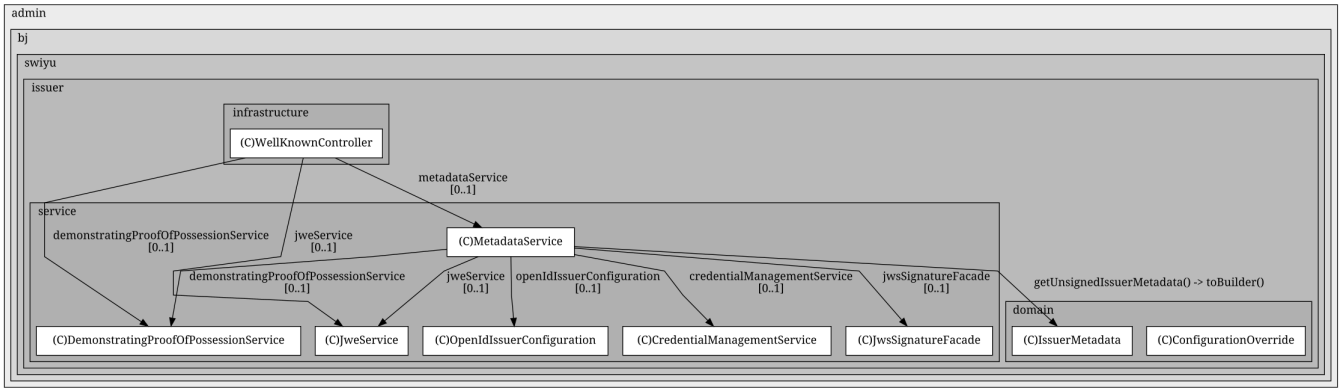
- Calls CredentialMetadataService to retrieve supported credential types and details.
- CredentialMetadataService in turn uses the domain object CredentialMetadata.



4.1.1.3. WellKnownController (infrastructure.web.signer)

Provides well-known / discovery endpoints.

- Uses MetadataService to build issuer/OpenID configuration.
- Uses DemonstratingProofOfPossessionService and JweService to expose crypto-related configuration and encrypted samples if needed.
- Indirectly works with domain objects like IssuerMetadata and ConfigurationOverride.



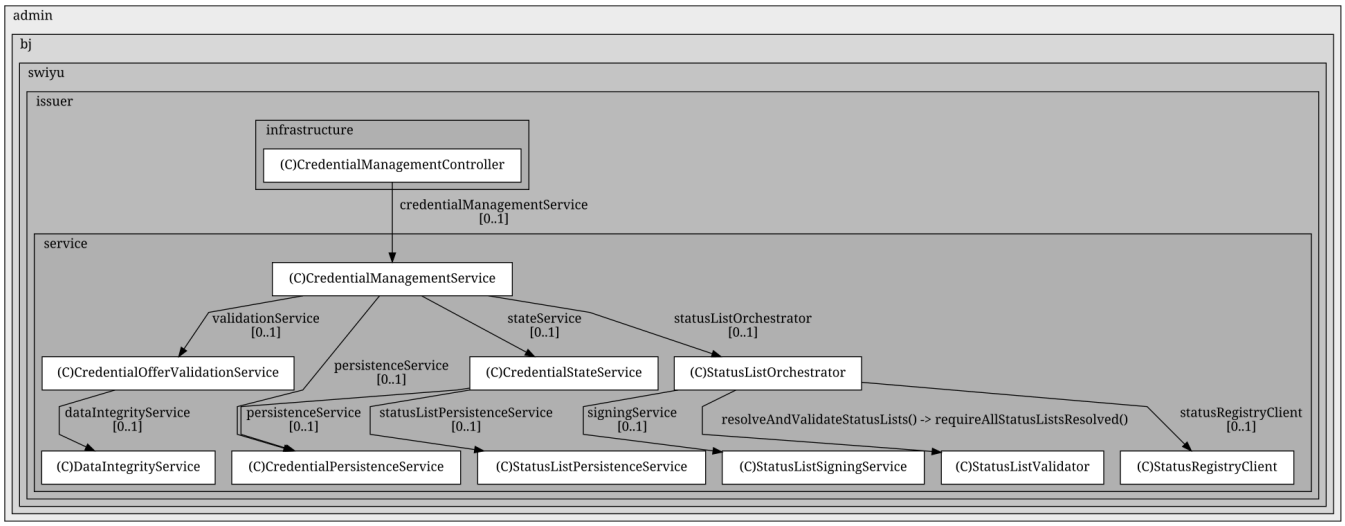
4.1.2. Management

4.1.2.1. CredentialManagementController (infrastructure.web.management)

Management API for credentials (operational / backoffice use).

Delegates to CredentialManagementService for:

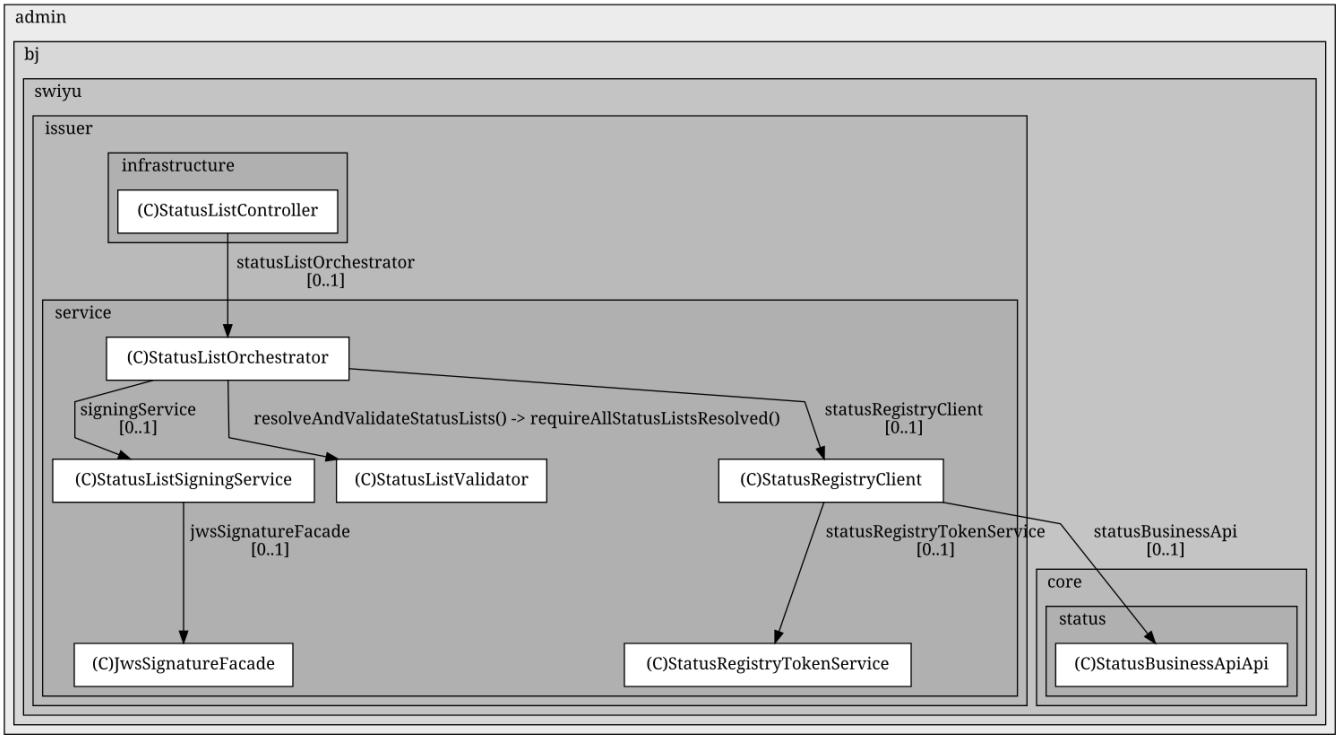
- Creating and updating credentials.
- Changing credential state (e.g. revocation/suspension).
- Triggering interactions with status lists.



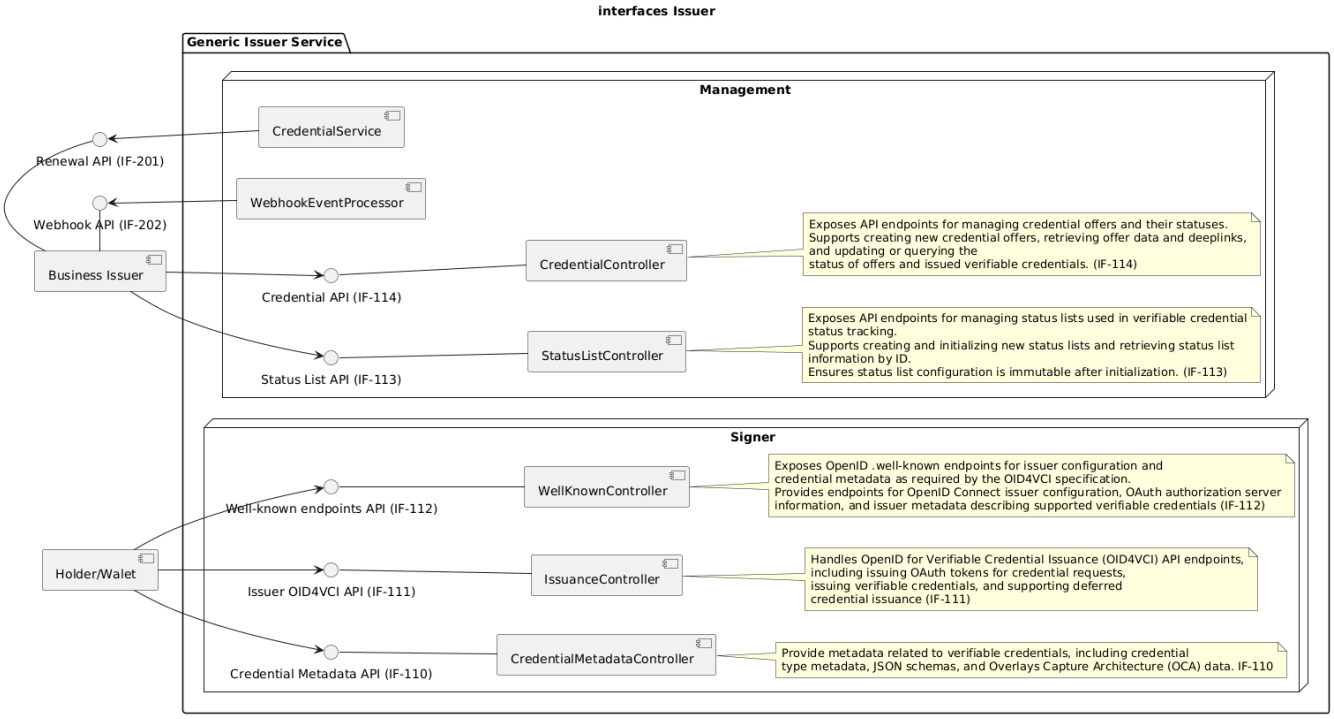
4.1.2.2. StatusListController (infrastructure.web.management)

Management and operational API for status lists.
Delegates to StatusListOrchestrator for:

- Creating, signing, and publishing status lists.
- Validating status lists before use.
- Synchronizing with external status registry.



5. Interfaces



6. Interface - Credential Management API Specification (IF-114)

6.1. Overview

The Credential Management API provides endpoints for managing verifiable credential offers and their lifecycle. This API enables issuers to create credential offers, retrieve offer information, manage credential statuses, and handle deferred issuance flows.

Base URL: `/management/api/credentials`

Protocol: REST (JSON over HTTP/HTTPS)

6.2. Data Model Summary

6.2.1. CreateCredentialOfferRequestDto

Request model for creating a new credential offer.

```
{
  "metadata_credential_supported_id": ["string (required)"],
  "credential_subject_data": "object or JWT string (required)",
  "credential_metadata": {
    "deferred": "boolean (optional, default: false)",
    "vct#integrity": "string (optional)",
    "vct_metadata_uri": "string (optional)",
    "vct_metadata_uri#integrity": "string (optional)"
  },
  "offer_validity_seconds": "integer (optional)",
  "deferred_offer_validity_seconds": "integer (optional)",
  "credential_valid_from": "ISO 8601 datetime (optional)",
  "credential_valid_until": "ISO 8601 datetime (optional)",
  "status_lists": ["string"],
  "configuration_override": {
    "issuer_did": "string (optional)",
    "hsm_alias": "string (optional)"
  },
  "status_lists": [ string ]
}
```

6.2.2. CredentialWithDeeplinkResponseDto

Response containing credential offer ID and deeplink.

```
{
  "management_id": "uuid",
  "offer_id": "uuid",
  "offer_deeplink": "openid-credential-offer://?credential_offer_uri=..."
}
```

6.2.3. CredentialManagementDto

Detailed information about a credential offer.

```
{
  "id": "uuid",
  "status": "CredentialStatusTypeDto (Enum)",
  "renewal_request_count": "integer",
  "renewal_response_count": "integer",
  "credential_offers": [
    { /* CredentialInfoResponseDto */ }
  ],
  "dpop_key": "string"
}
```

```
Field Descriptions:
id: UUID – Unique identifier for the credential management entry.
status: CredentialStatusTypeDto (Enum) – The overall status (see enum values below).
renewal_request_count: Integer – Number of renewal requests.
renewal_response_count: Integer – Number of renewal responses.
credential_offers: Array of CredentialInfoResponseDto – List of credential offers associated with this management
entry.
dpop_key: Jwk with the public key of the holder
```

6.2.4. CredentialInfoResponseDto

```
{
  "status": "CredentialStatusTypeDto (Enum)",
  "metadata_credential_supported_id": ["string"],
  "credential_metadata": { /* CredentialOfferMetadataDto */ },
  "holder_jwks": ["string"],
  "key_attestations": ["string"],
  "client_agent_info": { /* ClientAgentInfoDto */ },
  "offer_expiration_timestamp": "long (epoch seconds)",
  "deferred_offer_expiration_seconds": "integer",
  "credential_valid_from": "string (ISO 8601 datetime)",
  "credential_valid_until": "string (ISO 8601 datetime)",
  "offer_deeplink": "string",
  "vc_hash": ["string"]
}
```

Field Descriptions:
status: CredentialStatusTypeDto (Enum) – Status of this credential offer
metadata_credential_supported_id: Array of strings – Supported credential type identifiers.
credential_metadata: CredentialOfferMetadataDto – Metadata for the credential offer.
holder_jwks: Array of strings – Holder's public keys (JWKs).
key_attestations: Array of strings – Key attestation values.
client_agent_info: ClientAgentInfoDto – Information about the client agent (e.g., user agent, IP).
offer_expiration_timestamp: Long (epoch seconds) – Expiration timestamp for the offer.
deferred_offer_expiration_seconds: Integer – Validity period for deferred offers (in seconds).
credential_valid_from: String (ISO 8601 datetime) – Start of credential validity.
credential_valid_until: String (ISO 8601 datetime) – End of credential validity.
offer_deeplink: String – Deeplink for the credential offer.
vc_hash: List of Strings with Hashes/ Json Web Signatures, allowing a minimal tracing possibility in case of VC misuse

6.2.5. StatusResponseDto

Current status of a credential offer or issued credential.

```
{
  "status": "string"
}
```

6.2.6. UpdateStatusResponseDto

Confirmation of status update operation.

```
{
  "id": "uuid",
  "status": "string",
  "status_lists": ["uuid"]
}
```

6.2.7. CredentialStatusTypeDto (Enum)

Possible Status Values:

- INIT - Initial status when offer created
- OFFERED - an offer link has been created, and not yet redeemed by a holder.
- CANCELLED - the VC was revoked before being claimed.
- IN_PROGRESS - very short lived state, if the Holder has redeemed the one-time-code, but not yet gotten their credential. To allow a holder to retry fetching the vc set the state to offered.
- DEFERRED - the offer has been used and all necessary data from the wallet has been received but the credential is not yet issued. To use this state the credential metadata entry has to have deferred set to true.
- READY - Status set by the business issuer to continue the issuance of the credential for the deferred flow.
- ISSUED - the VC has been collected by the holder and is valid.
- SUSPENDED - the VC has been temporarily suspended. To unsuspend change state to issued.
- REVOKED - the VC has been revoked. This state is final and can not be changed.
- EXPIRED - the lifetime of the VC expired (not used yet)
- REQUESTED - When Wallet requested a credential renewal

6.3. Endpoints

6.3.1. Create Credential Offer

Create a new credential offer that can be collected by a holder.

Endpoint: `POST /management/api/credentials`

Request Body: CreateCredentialOfferRequestDto

Response: CredentialWithDeeplinkResponseDto

6.3.2. Get Credential Information

Retrieve cached offer data for a specific credential.

This endpoint returns the **CredentialManagementDto** of the corresponding **CredentialManagement** object.
This DTO contains all relevant information about the CredentialManagement itself as well as all associated **CredentialOffers**, including their current states.

For **renewal offers**, only the states **REQUESTED** and **ISSUED** are applicable. As no asynchronous processing is used for renewals, these offers are returned to the holder immediately and are directly set to **ISSUED**. Consequently, a **READY** state does not exist for renewal offers and does not need to be explicitly evaluated.

Endpoint: `GET /management/api/credentials/{credentialManagementId}`

Path Parameters:

- credentialManagementId (UUID, required): The unique identifier of the credential management object

Response: CredentialManagementDto

6.3.3. Get Offer Information

Retrieve cached offer data for a specific offer.

This endpoint returns the **CredentialInfoResponseDto** of the corresponding **CredentialOffer** object.
This DTO contains all relevant information about CredentialOffers offer, including their current states.

For **renewal offers**, only the states **REQUESTED** and **ISSUED** are applicable. As no asynchronous processing is used for renewals, these offers are returned to the holder immediately and are directly set to **ISSUED**. Consequently, a **READY** state does not exist for renewal offers and does not need to be explicitly evaluated.

Endpoint: `GET /management/api/credentials/{credentialManagementId}/offers/{credentialOfferId}`

Path Parameters:

- credentialManagementId (UUID, required): The unique identifier of the credential management object
- credentialOfferId (UUID, required): The unique identifier of the offer

Response: CredentialInfoResponseDto

`Get Credential Status`

This endpoint can still be used with the **Management ID** to retrieve the current status.

The relevant status information is provided either from the corresponding **CredentialOffer** or from the **CredentialManagement**, depending on the lifecycle state of the credential:

- As long as the credential has not yet been issued, the status information is derived from the existing **CredentialOffer** (only one offer can exist at this stage).
- Once the credential has been issued, the status information is obtained from the **CredentialManagement**, reflecting one of the following states: **ISSUED**, **SUSPENDED**, or **REVOKED**.

Retrieve the current status of an offer or issued verifiable credential.

Endpoint: `GET /management/api/credentials/{credentialManagementId}/status`

Path Parameters:

- credentialManagementId (UUID, required): The unique identifier of the credential

Response: StatusResponseDto

6.3.4. Get Credential Offer Status

This endpoint can be used with the **Management ID** and **Offer ID** to retrieve the current status.

Retrieve the current status of the offer.

Endpoint: `GET /management/api/credentials/{credentialManagementId}/offers/{credentialOfferId}/status`

Path Parameters:

- credentialManagementId (UUID, required): The unique identifier of the credential
- credentialOfferId (UUID, required): The unique identifier of the offer

Response: StatusResponseDto

6.3.5. Update Credential for Deferred Flow

Update credential offer data for deferred issuance and set status to ready.

Endpoint: `PATCH /management/api/credentials/{credentialManagementId}`

Path Parameters:

- credentialManagementId (UUID, required): The unique identifier of the credential offer

Request Body:
offerDataMap the credential subject data to apply to the deferred offer as Map<String, Object> offerDataMap

Example

```
{
    "givenName": "Jane",
    "familyName": "Doe",
    "dateOfBirth": "1990-01-15",
    "nationalId": "CHE-123456789"
}
```

Response: `UpdateStatusResponseDto`

6.3.6. Update Credential Status

Manually set the status of an offer or verifiable credential.

In this request, a webhook is also triggered. Through this webhook, the state of the Offer or the Management Offer is sent back to the Business Issuer. This depends on the current state of the Offer.

If the Management Offer is in a pre-issuance state (INIT), the webhook is first triggered with the status change of the Offer (in this case, there can only be one).

Afterwards, during the post-issuance process, a possible status change of the Management Offer is processed. If the status changes, the Management Offer status transition is then sent via webhook.

Endpoint: `PATCH /management/api/credentials/{credentialManagementId}/status`

Path Parameters:

- credentialManagementId (UUID, required): The unique identifier of the credential

Query Parameters:

- `credentialStatus` (enum, required): New status value
  - `CANCELLED - the VC was revoked before being claimed.`
  - `READY - Status set by the business issuer to continue the issuance of the credential for the deferred flow`
  - `SUSPENDED - the VC has been temporarily suspended. To unsuspend change state to issued.`
  - `REVOKED - the VC has been revoked. This state is final and can not be changed.`

Response: `UpdateStatusResponseDto`

6.4. Error Handling

6.4.1. Standard Error Response Format

```
{
  "error": "ERROR_CODE (optional, for OAuth/OID4VC errors)",
  "error_description": "Human-readable error description",
  "detail": "Additional error details (optional)",
  "trace_id": "Trace identifier for debugging (optional)"
}
```

6.4.2. HTTP Status Codes

Status Code	Description	When Used
`200 OK`	Request succeeded	Successful GET, POST, PATCH operations
`400 Bad Request`	Invalid input data	Validation errors, malformed JSON
`401 Unauthorized`	Missing or invalid authentication	Invalid or expired JWT token
`403 Forbidden`	Insufficient permissions	User lacks required permissions
`404 Not Found`	Resource not found	Credential ID doesn't exist
`409 Conflict`	State conflict	Invalid state transition attempt
`500 Internal Server Error`	Server-side error	Unexpected system errors
`503 Service Unavailable`	External service failure	StatusList service unavailable

6.4.3. Common Error Descriptions

Status Code	Error Description	Detail Examples
`400 Bad Request`	"Bad Request"	Validation errors, invalid credential data
`404 Not Found`	"Not Found"	"Credential with ID {id} not found"
`422 Unprocessable Entity`	"Unprocessable Entity"	Field validation errors (e.g., "field_name: must not be null")
`500 Internal Server Error`	"Internal Server Error"	Configuration errors, unexpected exceptions

6.5. Field & Enum Reference

6.5.1. UpdateCredentialStatusRequestTypeDto (Enum)

Valid values for credential status updates:

Value	Description	Allowed From States
`REVOKED`	Permanently revoke credential	`ISSUED`, `ACTIVE`
`SUSPENDED`	Temporarily suspend credential	`ISSUED`, `ACTIVE`
`ACTIVE`	Reactivate suspended credential	`SUSPENDED`
`EXPIRED`	Mark credential as expired	`ISSUED`, `ACTIVE`

6.5.2. Credential Types

Common credential types (extensible):

- `VerifiableCredential` - Generic verifiable credential
- `IdentityCredential` - Identity document
- `EducationCredential` - Educational qualification
- `EmploymentCredential` - Employment verification
- `LicenseCredential` - Professional license

6.6. Examples

6.6.1. Example 1: Complete Immediate Issuance Flow

Step 1: Create Credential Offer

```
POST /management/api/credentials
Content-Type: application/json

{
  "metadata_credential_supported_id": ["IdentityCredential"],
  "credential_subject_data": {
    "givenName": "John",
    "familyName": "Smith",
    "dateOfBirth": "1985-07-20",
    "nationality": "CH",
    "personalId": "CHE-1234567890"
  },
  "credential_valid_from": "2025-03-17T00:00:00Z",
  "credential_valid_until": "2030-12-31T23:59:59Z",
  "offer_validity_seconds": 86400,
  "deferred_offer_validity_seconds": 604800,
  "credential_metadata": {
    "deferred": false
  },
  "status_lists": [
    "$STATUS_REGISTRY_URL"
  ]
}
```

Response:

```
{
  "management_id": "f7e6d5c4-3b2a-1098-7654-fedcba987654",
  "offer_id": "59566ef1-abaa-4dd4-9561-06bcd7d14a35",
  "deeplink": "openid-credential-offer://?credential_offer_uri=https://issuer.admin.ch/offers/f7e6d5c4"
}
```

Step 2: Monitor Status

```
 GET /management/api/credentials/f7e6d5c4-3b2a-1098-7654-fedcba987654/status
```

Response:

```
{
  "status": "ISSUED"
}
```

6.6.2. Example 2: Deferred Issuance Flow

Step 1: Create Deferred Offer

```
POST /management/api/credentials
Content-Type: application/json

{
  "metadata_credential_supported_id": ["EmploymentCredential"],
  "credential_subject_data": {
    "employeeId": "EMP-2024-001",
  },
  "offer_validity_seconds": 86400,
  "deferred_offer_validity_seconds": 604800,
  "credential_metadata": {
    "deferred": true
  },
  "status_lists": [
    "$STATUS_REGISTRY_URL"
  ]
}
```

Response:

```
{
  "management_id": "9a8b7c6d-5e4f-3210-9876-543210fedcba",
  "offer_id": "99e54d8b-8208-499d-8add-4654d03e6fc8",
  "deeplink": "openid-credential-offer://?credential_offer_uri=https://issuer.admin.ch/offers/9a8b7c6d"
}
```

Step 2: Complete Background Check (External Process)

```
[Time passes while verification for issuance occurs...]
```

Step 3: Update with Complete Data

```
PATCH /management/api/credentials/9a8b7c6d-5e4f-3210-9876-543210fedcba
Content-Type: application/json

{
    "employeeId": "EMP-2024-001",
    "givenName": "Alice",
    "familyName": "Johnson",
    "position": "Senior Engineer",
    "startDate": "2024-01-15",
    "department": "Engineering",
    "salary": "120000",
}
```

Response:

```
{
  "id": "9a8b7c6d-5e4f-3210-9876-543210fedcba",
  "status": "ISSUED",
  "status_lists": ["18fa7c77-9dd1-4e20-a147-fb1bec146085"]
}
```

6.6.3. Example 3: Revoking a Credential

```
 PATCH /management/api/credentials/a1b2c3d4-5678-90ab-cdef-1234567890ab/status?credentialStatus=REVOKED
```

Response:

```
{
  "id": "a1b2c3d4-5678-90ab-cdef-1234567890ab",
  "status": "REVOKED",
  "status_lists": ["18fa7c77-9dd1-4e20-a147-fb1bec146085"]
}
```

6.6.4. Example 4: Retrieving Credential Information

```
GET /management/api/credentials/a1b2c3d4-5678-90ab-cdef-1234567890ab
Accept: application/json
```

Response:

```
{
  "status": "ISSUED",
  "metadata_credential_supported_id": ["VerifiableCredential"],
  "credential_metadata": {
    "deferred": false
  },
  "holder_jwks": [],
  "key_attestations": [],
  "client_agent_info": null,
  "offer_expiration_timestamp": 1710677400,
  "deferred_offer_expiration_seconds": 604800,
  "credential_valid_from": "2025-03-17T10:00:00Z",
  "credential_valid_until": "2025-12-31T23:59:59Z",
  "credential_request": null,
  "offer_deeplink": "openid-credential-offer://?credential_offer_uri=https://issuer.admin.ch/offers/a1b2c3d4"
}
```

6.6.5. Example 5: Error Handling

Invalid Credential Data:

```
POST /management/api/credentials
Content-Type: application/json

{
  "credentialType": "",
  "credentialSubjectData": null
}
```

Response: `400 Bad Request`

```
{
  "error_description": "Unprocessable Entity",
  "detail": "metadata_credential_supported_id: 'metadata_credential_supported_id' cannot be empty,
credential_subject_data: must not be null"
}
```

7. Interface - Status List Management API Specification (IF-113)

The Status List Management API provides endpoints for managing status lists used in verifiable credential status tracking.

Base URL: `/management/api/status-list`

7.1.1. Status List Data Models

7.1.1.1. StatusListCreateDto

Request model for creating and initializing a new status list.

```
{
  "type": "TOKEN_STATUS_LIST",
  "maxLength": 100000,
  "config": {
    "bits": 2
  },
  "configuration_override": {
    "issuer_did": "did:example:123456789abcdefghi (optional)",
    "verification_method": "did:example:123456789abcdefghi#keys-1 (optional)",
    "key_id": "hsm-key-01 (optional)",
    "key_pin": "**** (optional)"
  }
}
```

Field Descriptions:

- `type`: Technical type of the status list (required). Currently supported: `TOKEN_STATUS_LIST`
- `maxLength`: Maximum number of status entries in the list (required, minimum: 1)
    - Example: 100000 entries
    - Memory size depends on type and config
- `config`: Configuration parameters based on status list type (required)
    - `bits`: Number of bits per token (required for TOKEN_STATUS_LIST)
        - 1: Supports 2 states (VALID, REVOKE)
        - 2: Supports 3 states (VALID, REVOKE, SUSPEND)

        - 4 or 8: Reserved for future expansion (additional states). Currently only values 1 or 2 are effectively used by implementation.
    - `purpose`: Optional descriptive purpose (currently unused by TOKEN_STATUS_LIST but accepted by API)
- `configuration_override`: Override cryptographic / DID settings for this list (all fields optional)
    - `issuer_did`: DID to use instead of default issuer id
    - `verification_method`: Fully qualified DID verification method identifier
    - `key_id`: Identifier of signing key in HSM
    - `key_pin`: PIN protecting the key (not the HSM partition PIN)

Important Notes:

- Status lists can only be initialized **once**
- Type, configuration, and length **cannot be changed** after initialization
- Ensure the `maxLength` is sufficient for your use case (immutable)
- Increasing `bits` increases possible states per credential reference; only configure what's required

7.1.1.2. StatusListDto

Response model containing status list information.

```
{
  "id": "uuid",
  "statusRegistryUrl": "string",
  "type": "TOKEN_STATUS_LIST",
  "maxListEntries": 100000,
  "remainingListEntries": 99987,
  "version": "string"
  "config": {
    "bits": 2,
    "purpose": ""
  }
}
```

Field Descriptions:

- `id`: Unique identifier (UUID) of the status list
- `statusRegistryUrl`: URI of the status list in the external registry
- `type`: Technical type of the status list
- `maxListEntries`: Total capacity of the status list
- `remainingListEntries`: Number of unused status entries still available (calculated as `maxLength - used`)
- `version`: Schema/version string injected from configuration
- `config`: Effective configuration map (contains at least `bits` and optionally `purpose`)

7.1.1.3. StatusListUpdateDto (Request model for updating a status list)

Request model for updating and (re)publishing an existing status list and optionally updating the persisted signing / DID override configuration.
Example

```
{
  "configuration_override": {
    "issuer_did": "did:example:123456789abcdefghi (optional)",
    "verification_method": "did:example:123456789abcdefghi#keys-1 (optional)",
    "key_id": "hsm-key-01 (optional)",
    "key_pin": "**** (optional)"
  }
}
```

Field Descriptions

- *configuration_override (optional)*: Override cryptographic / DID settings for this status list (all fields inside are optional)
- *issuer_did (optional):* DID to use instead of the default issuer id
- *verification_method (optional)*: Fully qualified DID verification method identifier (e.g., did:...#key-1)
- *key_id (optional):* Identifier of signing key in HSM
- *key_pin (optional):* PIN protecting the key (not the HSM partition PIN)

7.1.2. Endpoints

7.1.2.1. Create and Initialize Status List

Create and initialize a new status list for credential status tracking.

Endpoint: `POST /management/api/status-list`

Request Body:

```
{
  "type": "TOKEN_STATUS_LIST",
  "maxLength": 100000,
  "config": {
    "bits": 2
  }
}
```

Response: `200 OK`

```
{
  "id": "18fa7c77-9dd1-4e20-a147-fb1bec146085",
  "statusRegistryUrl": "https://registry.example.com/status-lists/18fa7c77-9dd1-4e20-a147-fb1bec146085",
  "type": "TOKEN_STATUS_LIST",
  "maxListEntries": 100000,
  "remainingListEntries": 100000,
  "version": "1.0",
  "config": {
    "bits": 2,
    "purpose": ""
  }
}
```

Error Response: 500

```
{
  "error_description": "Internal Server Error",
  "detail": "Failed to create status list - caused by - Failed to lookup authorization token for accessing the
status registry. There was no token provided under the key 'STATUS_REGISTRY'."
}
```

Error Response: `404`

```
{
  "error_description": "Not Found",
  "detail": " "
}
```

(Thrown when a uniqueness / integrity constraint prevents creation — matches BadRequestException: Status list already initialized.)

Important: This operation can only be performed **once per status list**. Configuration becomes immutable immediately.

7.1.2.2. Get Status List Information

Retrieve information about a specific status list.

Endpoint: `GET /management/api/status-list/{statusListId}`

Path Parameters:

- `statusListId` (UUID, required): The unique identifier of the status list

Response: `200 OK`

```
{
  "id": "18fa7c77-9dd1-4e20-a147-fb1bec146085",
  "statusRegistryUrl": "https://registry.example.com/status-lists/18fa7c77-9dd1-4e20-a147-fb1bec146085",
  "type": "TOKEN_STATUS_LIST",
  "maxListEntries": 100000,
  "remainingListEntries": 99987,
  "version": "1.0",
  "config": {
    "bits": 2,
    "purpose": ""
  }
}
```

Error Response: `404 Not Found`

```
{
  "error_description": "Not Found",
  "detail": "Status list 18fa7c77-9dd1-4e20-a147-fb1bec146085 not found"
}
```

(Exact string pattern emitted by `ResourceNotFoundException` in service layer.)

7.1.2.3. Update Status List Registry Entry

Update (re)publishes the status list to the status registry and can optionally set/update the persisted configuration override used for signing (e.g., for HSM key selection).

Endpoint: `POST /management/api/status-list/{statusListId}`

Path Parameters:

- `statusListId` (UUID, required): The unique identifier of the status list

Request Body (optional)
Body schema: **StatusListUpdateDto**

If a configuration_override is provided, it is merged into the currently stored override (non-null fields win) and then persisted on the status list. The persisted override is used for subsequent publications.

Response: `200 OK`

```
{
  "id": "18fa7c77-9dd1-4e20-a147-fb1bec146085",
  "statusRegistryUrl": "https://registry.example.com/status-lists/18fa7c77-9dd1-4e20-a147-fb1bec146085",
  "type": "TOKEN_STATUS_LIST",
  "maxListEntries": 100000,
  "remainingListEntries": 99987,
  "version": "1.0",
  "config": {
    "bits": 2,
    "purpose": ""
  }
}
```

Error Response (not found): `404 Not Found`

```
{
  "error_description": "Not Found",
  "detail": "Status list 18fa7c77-9dd1-4e20-a147-fb1bec146085 not found"
}
```

Important Notes

- This endpoint can be used regardless of whether automatic status list synchronization is enabled.
- If configuration_override is provided:
  - it is merged into the existing persisted override (non-null fields win),
  - the result is persisted on the status list,
  - and used for subsequent publications (e.g., selecting signing key material in an HSM setup)
- If the request body is empty/missing, the endpoint republishes using the currently persisted configuration.

7.1.3. Status List Usage Examples

7.1.3.1. Example 1: Create a Token Status List with Revocation and Suspension

Request:

```
POST /management/api/status-list
Content-Type: application/json

{
  "type": "TOKEN_STATUS_LIST",
  "maxLength": 100000,
  "config": {
    "bits": 2
  }
}
```

Response:

```
{
  "id": "18fa7c77-9dd1-4e20-a147-fb1bec146085",
  "statusRegistryUrl": "https://registry.admin.ch/status-lists/18fa7c77-9dd1-4e20-a147-fb1bec146085",
  "type": "TOKEN_STATUS_LIST",
  "maxListEntries": 100000,
  "remainingListEntries": 100000,
  "version": "1.0",
  "config": {
    "bits": 2,
    "purpose": ""
  }
}
```

7.1.3.2. Example 2: Create a Token Status List with Revocation Only

Request:

```
POST /management/api/status-list
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...

{
  "type": "TOKEN_STATUS_LIST",
  "maxLength": 50000,
  "config": {
    "bits": 1
  }
}
```

Response:

```
{
  "id": "a2b3c4d5-1234-5678-90ab-cdef12345678",
  "statusRegistryUrl": "https://registry.admin.ch/status-lists/a2b3c4d5-1234-5678-90ab-cdef12345678",
  "type": "TOKEN_STATUS_LIST",
  "maxListEntries": 50000,
  "remainingListEntries": 50000,
  "version": "1.0",
  "config": {
    "bits": 1,
    "purpose": ""
  }
}
```

7.1.3.3. Example 3: Check Status List Capacity

Request:

```
GET /management/api/status-list/18fa7c77-9dd1-4e20-a147-fb1bec146085
```

Response:

```
{
  "id": "18fa7c77-9dd1-4e20-a147-fb1bec146085",
  "statusRegistryUrl": "https://registry.admin.ch/status-lists/18fa7c77-9dd1-4e20-a147-fb1bec146085",
  "type": "TOKEN_STATUS_LIST",
  "maxListEntries": 100000,
  "remainingListEntries": 87543,
  "version": "1.0",
  "config": {
    "bits": 2,
    "purpose": ""
  }
}
```

Interpretation: Remaining entries = `maxListEntries - used`; used entries are determined by current credential statuses referencing the list.

7.1.3.4. Example 4: Manually Update Status List Registry

Precondition: Automatic synchronization disabled in config.

Request:

```
POST /management/api/status-list/18fa7c77-9dd1-4e20-a147-fb1bec146085
```

Response:

```
{
  "id": "18fa7c77-9dd1-4e20-a147-fb1bec146085",
  "statusRegistryUrl": "https://registry.admin.ch/status-lists/18fa7c77-9dd1-4e20-a147-fb1bec146085",
  "type": "TOKEN_STATUS_LIST",
  "maxListEntries": 100000,
  "remainingListEntries": 87543,
  "version": "1.0",
  "config": {
    "bits": 2,
    "purpose": ""
  }
}
```

8. Interface - Well-known Endpoints API Specification (IF-112)

8.1. Overview

This specification documents the OpenID/OID4VCI well-known endpoints exposed by the issuer, implemented by `WellKnownController`.

These endpoints provide:

- OpenID Connect authorization server configuration
- OpenID4VCI issuer metadata describing supported verifiable credentials
- Tenant-scoped variants for multi-tenant deployments
- Optional JWT-wrapped signed responses when requested via the `Accept` header

Base paths:

- Global: `/oid4vci/.well-known/*` and `/.well-known/*`
- Tenant-scoped: `/oid4vci/{tenantId}/.well-known/*` and `/{tenantId}/.well-known/*`

8.2. Headers

8.2.1. Required/Recommended

- `Accept`: Determines the response format
  - `application/json` (default): Returns structured JSON objects
  - `application/jwt`: Returns compact JWS (signed JWT) string per endpoint used for signed metadata
- `Authorization`: `Bearer <token>` (if required by deployment policy)
- `Content-Type`: Not applicable for GET endpoints

8.2.2. Notes

- For tenant-scoped endpoints, `Accept: application/jwt` is respected for both OpenID configuration and issuer metadata.
- For global endpoints, `Accept: application/jwt` is supported for issuer metadata; OpenID configuration uses JSON unless otherwise implemented.

8.3. Data Models

These models reflect the controller's return types and behavior and are aligned precisely to the implementation.

8.3.1. OAuthAuthorizationServerMetadataDto (JSON)

Record fields:

- `issuer` (string, required)
- `token_endpoint` (string, required)
- `dpop_signing_alg_values_supported` (array of string, optional)

Notes:

- `dpop_signing_alg_values_supported` is injected by `DemonstratingProofOfPossessionService.addSigningAlgorithmsSupported(...)`.

Example:

```
{
  "issuer": "https://issuer.example.com",
  "token_endpoint": "https://issuer.example.com/oauth2/token",
  "dpop_signing_alg_values_supported": ["ES256"]
}
```

8.3.2. IssuerMetadata (JSON)

Object fields:

- `credential_issuer` (string, required): The Credential Issuer's identifier
- `authorization_servers` (array of string, optional)
- `credential_endpoint` (string, required, must end with "/credential")
- `nonce_endpoint` (string, optional, must end with "/nonce")
- `deferred_credential_endpoint` (string, optional)
- `notification_endpoint` (string, optional, currently not supported; size constraint enforces empty)
- `credential_configurations_supported` (object, required, non-empty): Map<string, CredentialConfiguration>
- `credential_request_encryption` (object, optional): IssuerCredentialRequestEncryption
- `credential_response_encryption` (object, optional): IssuerCredentialResponseEncryption

- `batch_credential_issuance` (object, optional): BatchCredentialIssuance
- `version` (string, required): Must match regex `^1\.0$`
- `display` (array of MetadataIssuerDisplayInfo, optional)

Notes:

- Encryption options are provided via `EncryptionService.issuerMetadataWithEncryptionOptions()`.
- The controller unwraps Spring proxy/cached instances via `AopProxyUtils.getSingletonTarget(...)`.

Example:

```
{
  "credential_issuer": "https://issuer.example.com",
  "authorization_servers": ["https://issuer.example.com"],
  "credential_endpoint": "https://issuer.example.com/credential",
  "nonce_endpoint": "https://issuer.example.com/nonce",
  "deferred_credential_endpoint": "https://issuer.example.com/deferred/credential",
  "credential_configurations_supported": {
    "IdentityCredential": {
      "format": "jwt_vc_json",
      "cryptographic_binding_methods_supported": ["jwk"],
      "cryptographic_suites_supported": ["ES256"],
      "credential_metadata": {
        "display": [...],
        "claims": [...]
      }
    }
  },
  "credential_request_encryption": {
    "jwks": {"keys": []},
    "enc_required": true
  },
  "credential_response_encryption": {
    "enc_required": true
  },
  "version": "1.0",
  "display": [{"name": "Issuer", "locale": "en"}]
}
```

8.3.3. Signed response (JWT)

When the endpoint supports signed responses and the `Accept` header is matched as described above, the response is a compact JWS string:

```
<base64url(header)>.<base64url(payload)>.<base64url(signature)>
```

Example (shortened):

```
eyJraWQiOiJkaWQ6Li4uIiwidHlwIjoiSldUIiwiYWxnIjoiRVMyNTYifQ.
eyJpc3MiOiJodHRwczovL2lzc3Vlci5leGFtcGxlLmNvbSIsImNyZWRlbnRpYWxzX3N1cHBvcnRlZCI6W10uLi59.c2lnbmF0dXJl
```

8.4. Endpoints

8.4.1. Global OpenID Configuration

Provide OpenID Connect and OID4VCI relevant configuration.

- GET `/oid4vci/.well-known/openid-configuration`
- GET `/.well-known/openid-configuration`
- GET `/oid4vci/.well-known/oauth-authorization-server`
- GET `/.well-known/oauth-authorization-server`

Response formats:

- `Accept: application/json` (default) `OAuthAuthorizationServerMetadataDto` (JSON)

Response: `200 OK`

```
{
  "issuer": "https://issuer.example.com",
  "token_endpoint": "https://issuer.example.com/oauth2/token",
  "dpop_signing_alg_values_supported": ["ES256"]
}
```

Errors: see Default Exception Handling

8.4.2. Global Issuer Metadata

Provide issuer metadata describing credentials that can be issued.

- GET `/oid4vci/.well-known/openid-credential-issuer`
- GET `/.well-known/openid-credential-issuer`

Response formats:

- `Accept: application/json` (default) `IssuerMetadata` (JSON)

Response: `200 OK` (JSON example)

```
{
  "credential_issuer": "http://localhost:8080",
  "credential_endpoint": "http://localhost:8080/oid4vci/api/credential",
  "nonce_endpoint": "http://localhost:8080/oid4vci/api/nonce",
  "credential_configurations_supported": {
    "university_example_sd_jwt": {
      "format": "vc+sd-jwt",
      "vct": "urn:vct:com.example.credential",
      "credential_metadata": {
        "claims": [
           {
             "path": ["type"]
             "mandatory": true,
           },
           {
             "path": ["name"],
             "mandatory": true
           }
        ]
      },
      // legacy claims start
      "claims": {
        "type": {
          "mandatory": true,
          "value_type": "string"
        },
        "name": {
          "mandatory": true,
          "value_type": "string"
        }
      },
      // legacy claims end
      "cryptographic_binding_methods_supported": [
        "jwk"
      ],
      "credential_signing_alg_values_supported": [
        "ES256"
      ],
      "proof_types_supported": {
        "jwt": {
          "proof_signing_alg_values_supported": [
            "ES256"
          ]
```

```
        }
      }
    }
  },
  "credential_request_encryption": {
    "enc_values_supported": [
      "A128GCM"
    ],
    "encryption_required": false,
    "jwks": {
      "keys": [
        {
          "kty": "EC",
          "crv": "P-256",
          "kid": "224f67f8-be32-4e7b-b185-6c136c915dfa",
          "x": "G6vZ0bZdrxKMjpJtU-yCmiIvyquAzfVLmtrsOVp_z1Y",
          "y": "_ZP8XR80C6SslLinsLiWlEZ3RXNan0HqGs3aHX4q47k"
        }
      ]
    }
  },
  "credential_response_encryption": {
    "enc_values_supported": [
      "A128GCM"
    ],
    "encryption_required": false
  },
  "version": "1.0"
}
```

Response: `200 OK` (JWT example)

```
eyJraWQiOiJkaWQ6Li4uIiwidHlwIjoiSldUIiwiYWxnIjoiRVMyNTYifQ.eyJjcmVkZW50aWFsc19zdXBwb3J0ZWQiOltdfQ.sig
```

Errors: see Default Exception Handling

8.4.3. Tenant-scoped Issuer Metadata

Provide issuer metadata for a specific tenant.

- GET `/oid4vci/{tenantId}/.well-known/openid-credential-issuer`
- GET `/{tenantId}/.well-known/openid-credential-issuer`

Path parameters:

- `tenantId` (UUID, required): Tenant identifier

Response formats:

- `Accept: application/json` (default) `IssuerMetadata` (JSON)
- `Accept: application/jwt` (exact match) signed JWT (compact JWS)

Response: `200 OK` (JSON example)

```
{
  "credential_issuer": "https://issuer.example.com/tenants/18fa7c77-9dd1-4e20-a147-fb1bec146085",
  "credential_endpoint": "https://issuer.example.com/credential",
  "credential_configurations_supported": {
    "VerifiableCredential": {
      "format": "jwt_vc_json"
    }
  },
  "version": "1.0"
}
```

Response: `200 OK` (JWT example)

```
eyJraWQiOiJkaWQ6Li4uIiwidHlwIjoiSldUIiwiYWxnIjoiRVMyNTYifQ.
eyJ0ZW5hbnRJZCI6IjE4ZmE3Yzc3LTlkZDEtNGUyMC1hMTQ3LWZiMWJlYzE0NjA4NSJ9.sig
```

Errors: see Default Exception Handling

8.4.4. Tenant-scoped OpenID Configuration

Provide OpenID configuration for a specific tenant.

- GET `/oid4vci/{tenantId}/.well-known/openid-configuration`
- GET `/{tenantId}/.well-known/openid-configuration`
- GET `/oid4vci/{tenantId}/.well-known/oauth-authorization-server`
- GET `/{tenantId}/.well-known/oauth-authorization-server`

Path parameters:

- `tenantId` (UUID, required): Tenant identifier

Response formats:

- `Accept: application/json` (default) `OAuthAuthorizationServerMetadataDto` (JSON)
- `Accept` header that starts with `application/jwt` signed JWT (compact JWS)

Response: `200 OK` (JSON example)

```
{
  "issuer": "https://issuer.example.com/tenants/18fa7c77-9dd1-4e20-a147-fb1bec146085",
  "token_endpoint": "https://issuer.example.com/oauth2/token",
  "dpop_signing_alg_values_supported": ["ES256"]
}
```

Response: `200 OK` (JWT example)

```
eyJraWQiOiJkaWQ6Li4uIiwidHlwIjoiSldUIiwiYWxnIjoiRVMyNTYifQ.
eyJpc3MiOiJodHRwczovL2lzc3Vlci5leGFtcGxlLmNvbS90ZW5hbnRzLzE4ZmE3YyJ9.sig
```

Errors: see Default Exception Handling

8.5. Content Negotiation

- `application/json`: returns typed JSON objects (`OAuthAuthorizationServerMetadataDto`, `IssuerMetadata`).
- `application/jwt`: for tenant endpoints only, returns signed compact JWS string when supported.
  - Issuer metadata: only when `Accept` equals `application/jwt`.
  - OpenID configuration: when `Accept` starts with `application/jwt`.

If the `Accept` header is not present or does not match, JSON is returned by default.

8.6. Default Exception Handling

All endpoints use the application-wide default exception handler. Error responses follow this structure (aligned with your existing management API spec):

```
{
  "error": "ERROR_CODE (optional)",
  "error_description": "Human-readable error description",
  "detail": "Additional error details (optional)",
  "trace_id": "Trace identifier (optional)"
}
```

HTTP status codes:

- 200 OK: Successful response
- 400 Bad Request: Invalid parameters, malformed Accept header, etc.
- 401 Unauthorized: Missing or invalid authentication (if enforced)
- 403 Forbidden: Insufficient permissions (if enforced)
- 404 Not Found: Tenant not found or resource unavailable
- 500 Internal Server Error: Unexpected server errors

Examples:

- `404 Not Found`

```
{
  "error_description": "Not Found",
  "detail": "Tenant with ID 18fa7c77-9dd1-4e20-a147-fb1bec146085 not found"
}
```

9. Interface - Issuance Controller API Specification (IF-111)

Public OpenID for Verifiable Credential Issuance (OID4VCI) API endpoints, including issuing OAuth tokens for credential requests, issuing verifiable credentials, and supporting deferred credential issuance (IF-111), implemented by IssuanceController.

These endpoints provide:

- POST /token — issues OAuth access tokens (supports form-encoded and deprecated param-based variant)
  POST /nonce — returns a self-contained nonce (and sets a DPoP nonce header)
  POST /credential — main credential issuance endpoint (immediate issuance)
  POST /deferred_credential — credential retrieval for deferred issuance (uses transaction id)

Base paths:

- Global: /oid4vci/api

9.1. Headers

9.1.1. Required/Recommended

- `Accept`: Determines the response format
  - `application/json` (default): Returns structured JSON objects
  - `application/jwt`: Returns compact JWS (signed JWT) string per endpoint
- `Authorization`: `Bearer <token>` (if required by deployment policy)
- `Content-Type`: Not applicable for GET endpoints

9.1.2. Notes

- For tenant-scoped endpoints, `Accept: application/jwt` is respected for both OpenID configuration and issuer metadata.
- For global endpoints, `Accept: application/jwt` is supported for issuer metadata; OpenID configuration uses JSON unless otherwise implemented.

9.2. Data Models

These models reflect the controller's return types and behavior and are aligned precisely to the implementation.

9.2.1. NonceRespone (JSON)

Record fields:

- c_nonce String containing an unpredictable challenge to be used when creating a proof of possession of the key.

Exapmle:

```
{
        "c_nonce": "c9c737f3-f37b-4331-a23c-313dc821fac5::2025-08-07T16:30:19.021045078Z"
}
```

9.2.2. OAuthToken Response (JSON)

Object fields:

- access_token String containing an unpredictableaccess token
- refresh_token String token used for token refresh
- expiresIn Long the number of seconds that the access token will be valid
- c_nonce DEPRECATED! since OID4VCI 1.0, String
- `token_type: String is always "BEARER"`

Example:

```
{
    "access_token": "your ACCESS_TOKEN",
    "refresh_token": "your REFRESH_TOKEN",
    "expires_in": 600,
    "token_type": "BEARER | DPoP"
}
```

9.2.3. CreateCredentialRequestDto (JSON)

Request to the Credential Endpoint as defined in OID4VCI 1.0 specification

Record fields:

- credential_configuration_id (mandatory) String that uniquely identifies one of the keys in the name/value pairs stored in the credential_configurations_supported Credential Issuer metadata. Example = "university_example_sd_jwt")
- proofs (optional) providing 1+ proof of possessions of the cryptographic key material to which the issued Credential instances will be bound to
- credential_response_encryption (optional) providing information how to encrypt the Credential Response, if present.

Example request:

```
{
    "credential_configuration_id": "university_example_sd_jwt",
    "proofs": {
        "jwt": [

"eyJ0eXAiOiJvcGVuaWQ0dmNpLXByb29mK2p3dCIsImFsZyI6IkVTMjU2IiwiandrIjp7Imt0eSI6IkVDIiwidXNlIjoic2lnIiwiY3J2IjoiUC0
yNTYiLCJraWQiOiJUZXN0LUtleS0wIiwieCI6Ii1QRXVmRjdnZmRta0dHVGQ5cG95YkdqdmkzSUxpeWFSdDNHN0F5aWxOSmsiLCJ5IjoianMyRDN
HRzNsdEZEZGplbG1yYzVWby0tejFoTnJJT3VVdFFtTGs1RkozTSIsImlhdCI6MTc1NDU4NDE3OH19.
eyJhdWQiOiJodHRwOi8vbG9jYWxob3N0OjgwODAvb2lkNHZjaSIsIm5vbmNlIjoiYzljNzM3ZjMtZjM3Yi00MzMxLWEyM2MtMzEzZGM4MjFmYWM1
OjoyMDI1LTA4LTA3VDE2OjMwOjE5LjAyMTA0NTA3OFoiLCJpYXQiOjE3NTQ1ODQzMTl9.tq_ZvRbz7BoGOdMrPHwSw64gI-
wzqQ7sD4XfjdKIW3iL8rj7FX-s8Fpsed0C8qITUl3K_8UWRfeoLZwA85NKuA"
        ]
    },
    "credential_response_encryption": {
                "alg": "ECDH-ES",
        "enc": "A128GCM",
        "jwk": {"kty":"EC","crv":"P-256","kid":"transportEncKeyEC","x":"
DTaouFJpyVkLvfhoOvuTDR6_nmTt7YTvEHsHzK0Ingk","y":"vOipfo61Sy64XpneRyR5g6NCGXLv_Q7f3-kEDMT-G9U"
    }
}
```

9.2.4. CredentialEndpointResponse (JSON)

Record fields:

- credentials OPTIONAL. Contains an array of one or more issued Credentials. MUST NOT be used if the transaction_id parameter is present.
- transactionId String identifying a Deferred Issuance transaction
- interval REQUIRED if transaction_id is present. Contains a positive number that represents the minimum amount of time in seconds that the Wallet

Example for non-deferred (direct) issuance:

```
{
    "credentials":
["eyJ2ZXIiOiIxLjAiLCJ0eXAiOiJ2YytzZC1qd3QiLCJhbGciOiJFUzI1NiIsImtpZCI6ImRpZDpleGFtcGxlOmxvY2FsaG9zdCUzQTgwODA6YWJjYWJjI3Nkand0In0.
eyJfc2QiOlsiMkVHOG55NjVPY1RjMDYxSGtUbW43M0NJV3hHT2xFVVpMbUkxc3B2NVNmMCIsInNKNXNaZ2wyU1VIU18zOGJwOXJYMnpHQ0hjcHlXQ0EtcUpOaE1UWlZvXzgiLCJ6bW5HRUdTd3NWdlNhVFpPZ18zR0Q1WHIybzdwS2FzRnF6WkxtRGQzT2lvIl0sInZjdCNpbnRlZ3JpdHkiOiJzaGEyNTYtU1ZITGZLZmNaY0JydytkOUVMLzFFWHh2R0Nka1E3dE1HdlptZDB5c01jaz0iLCJuYmYiOjE3NTQ1ODEzNzgsInZjdCI6Imh0dHA6Ly9sb2NhbGhvc3Q6ODA4MC9vaWQ0dmNpL3ZjdC9teS12Y3QtdjAxIiwiX3NkX2FsZyI6InNoYS0yNTYiLCJpc3MiOiJkaWQ6dGR3OmV4YW1wbGUiLCJjbmYiOnsia3R5IjoiRUMiLCJ1c2UiOiJzaWciLCJjcnYiOiJQLTI1NiIsImtpZCI6IlRlc3QtS2V5IiwieCI6ImZ5THhPVlpKak52dW53UTJfLWdyZzFqVnBJYzVkWEhHcHBSVDVRdVVXSTQiLCJ5IjoibFgxdktFOXl0QXQyRlNrNEpXY3Bxb1RvNDltbnYwam9rQ2gxRld1YTJqayIsImlhdCI6MTc1NDU4MTM4OCwiandrIjp7Imt0eSI6IkVDIiwidXNlIjoic2lnIiwiY3J2IjoiUC0yNTYiLCJraWQiOiJUZXN0LUtleSIsIngiOiJmeUx4T1ZaSmpOdnVud1EyXy1ncmcxalZwSWM1ZFhIR3BwUlQ1UXVVV0k0IiwieSI6ImxYMXZLRTl5dEF0MkZTazRKV2NwcW9UbzQ5bW52MGpva0NoMUZXdWEyamsiLCJpYXQiOjE3NTQ1ODEzODh9fSwiZXhwIjoxNzU0NTgxNTA4LCJpYXQiOjE3NTQ1ODE0MTIsInN0YXR1cyI6eyJzdGF0dXNfbGlzdCI6eyJ0eXBlIjoiU3dpc3NUb2tlblN0YXR1c0xpc3QtMS4wIiwidXJpIjoiaHR0cHM6Ly9sb2NhbGhvc3Q6ODA4MC9zdGF0dXMiLCJpZHgiOjB9fX0.
iTC8TZEJAOoE4DuWBtNSsSsy3dhaVmzXhql6k_K0pCVJszMBnEc0vbTszZYcjfwB2svriZGXBAiGZzgZZJ6Psg~WyJTYXRZVGQ2X1UwZWJ2NkZIWXJpQ1Z3IiwiYXZlcmFnZV9ncmFkZSIsIjUuMzMiXQ~WyJTb3dTSk1PVWdZMm5qSnlNZnRGdER3IiwiZGVncmVlIiwiQmFjaGVsb3Igb2YgU2NpZW5jZSJd~WyJCa3YwZ3BiTnFwYVA5TzZRay04VVZnIiwibmFtZSIsIkRhdGEgU2NpZW5jZSJd~"]
}
```

9.2.5. DeferredCredentialEndpointRequest (JSON)

`RequestDto to the deferred credential endpoint`

Record fields:

- transaction_id Id received from the create credential request for the deferred flow

Example:

```
{
        "transaction_id": "3c2e23f5-6aea-4478-b4a0-304fdc1b4933",
    "credential_response_encryption": {
      "jwk": { ... },
      "enc": "string"
    }
}
```

9.2.6. Proofs(JSON)

`ProofsDto represents the proofs object in the OID4VCI Credential Request.`

Record fields:

- `jwt -> List<String> with proofs`

  present. Contains a positive number that represents the minimum amount of time in seconds that the Wallet

Example for non-deferred (direct) issuance:

```
 "jwt": [

"eyJ0eXAiOiJvcGVuaWQ0dmNpLXByb29mK2p3dCIsImFsZyI6IkVTMjU2IiwiandrIjp7Imt0eSI6IkVDIiwidXNlIjoic2lnIiwiY3J2IjoiUC0
yNTYiLCJraWQiOiJUZXN0LUtleSIsIngiOiJrdHFJRFpoUjFmY2NlM3VGanpxdDdLRVlEdVdweFJoX3pqdkszanZsS2k4IiwieSI6Ik1UV2ZObTJ
6dy1CbklqM2szbW0xZVB3Q3hqTm9DSEowdXN6V25MeHVDemsiLCJpYXQiOjE3NTMyNjkyNzZ9fQ.
eyJhdWQiOiJodHRwOi8vbG9jYWxob3N0OjgwODAvb2lkNHZjaSIsIm5vbmNlIjoiY2U5YzEzNzgtODc2Yi00OGUyLTg0ZmUtOGE0ZjUwZGFkZmJm
IiwiaWF0IjoxNzUzMjY5Mjc2fQ.ck-
6Oq6IAav1VdFOkq9Qh7tzrl52jJvFBU3aPcZ_20oE73Cf4izN0ECmmiJm_qUMvYJlykQFsX2sW43gFC6vCw",
        "..."
]
```

9.3. Endpoints

9.3.1. Access Token / Refresh Token - Endpoint

- POST /oid4vci/api/token

9.3.1.1. Request:

Consumes:

- `application/x-www-form-urlencoded`. With body:
    - DPOP_HTTP_HEADER (optional, needed in refresh) String dpop,
    - grant_type String grantType,
    - pre-authorized_code required in preauth flow (not needed in refresh) String preAuthCode,
    - refresh_token (needed in refresh flow) String refreshToken,

Example header:

```
{
        "DPoP": "eyJ0eXAiOiJkcG9wK2p3dCIsImFsZyI6IkVTMjU2IiwiandrIjp7Imt0eSI6IkVDIiwiY3J2IjoiUC0yNTYiL...
3zCRvvT62tXSwLIOQ2qCXwWQeu_xaAAEvzG17rOxfQ",
        "Accept": "application/json"
}
```

With Decoded DPoP Header:

- typ: Must be "dpop+jwt"
- alg: Must be "ES256"
- jwk: The public key chosen by the client in JSON Web Key

```
{
  "typ": "dpop+jwt",
  "alg": "ES256",
  "jwk": {
    "kty": "EC",
    "crv": "P-256",
    "x": "09Wfv2o6-y0l7tgtK55KIGYYHoPrNhv21AIEcwnc5KA",
    "y": "ayC6Q00D874wvRlQ2BFcgEiYpO3jO-xfcqgJAgM5zGQ"
  }
}
```

With Decoded DPoP Payload:

- jti Unique Identifier for the JWT
- htm HTTP Method of the request (must be "POST" here)
- htu HTTP Target URI of the request (must point to the Token-Endpoint for example: `http://localhost:8080/oid4vci/api/token`)
- iat: Creation Timestamp of the JWT
- nonce: Nonce ("c_nonce") from the nonce endpoint

```
{
  "jti": "...",
  "htm": "POST",
  "htu": "http://localhost:8080/oid4vci/api/token",
  "iat": 1779096215,
  "nonce": "..."
}
```

Response formats:

- `Accept: application/json` (default) OauthToken

9.3.1.2. Response: `200 OK`

```
{
    "access_token": "your ACCESS_TOKEN",
    "refresh_token": "your REFRESH_TOKEN",
    "expires_in": 600,
    "token_type": "BEARER | DPoP"
}
```

Errors: see Default Exception Handling

9.3.2. Nonce Endpoint

Provides nonces for proof of possessions. For more information see Nonce Endpoint specification and RFC9449 for more details towards demonstrating proof of possession

- POST /oid4vci/api/nonce

Response formats:

- `Accept: application/json` (default) NonceResponse

Response: `200 OK`

```
{
    "c_nonce": "c9c737f3-f37b-4331-a23c-313dc821fac5::2025-08-07T16:30:19.021045078Z"
}
```

Errors: see Default Exception Handling

9.3.3. Credential Endpoint

Collects a credential request associated with a bearer token and issues a verifiable credential (immediate or deferred). Returns the issued credential as JSON or as a JWT depending on the type of credential and if encryption details are provided.

- POST /oid4vci/api/credential

9.3.3.1. Request:

Headers:

- Authorization (required) "Bearer <access_token>"
- DPoP (optional) — DPoP proof header (for demonstrating proof of possession)
- `Accept: application/json` (default)

- (optional / default) SWIYU-API-Version = 2

Example Header:

```
{
  "authorization": "Bearer 3d726fad-f748-432d-91f1-4401d68af016",
  "DPoP": "eyJ0eXAiOiJkcG9wK2p3dCIsImFsZyI6IkVTMjU2IiwiandrIjp7Imt0eSI....vBY7uVgC6MHmA"
}
```

With Decoded DPoP Header:

- typ Must be "dpop+jwt"
- alg Must be "ES256"
- jwk The public key chosen by the client in JSON Web Key (always the same)

```
{
  "typ": "dpop+jwt",
  "alg": "ES256",
  "jwk": {
    "kty": "EC",
    "crv": "P-256",
    "x": "09Wfv2o6-y0l7tgtK55KIGYYHoPrNhv21AIEcwnc5KA",
    "y": "ayC6Q00D874wvRlQ2BFcgEiYpO3jO-xfcqgJAgM5zGQ"
  }
}
```

With Decoded DPoP Payload:

- jti Unique Identifier for the JWT
- htm HTTP Method of the request (must be "POST" here)
- htu HTTP Target URI of the request (must point to the Token-Endpoint for example: `http://localhost:8080/oid4vci/api/credential`)
- iat Creation Timestamp of the JWT
- nonce Nonce ("c_nonce") from the nonce endpoint
- ath Hash of the access token. Must be base64url encoded with SHA-256

```
{
  "jti": "...",
  "htm": "POST",
  "htu": "http://localhost:8080/oid4vci/api/credential",
  "iat": 1779096215,
  "nonce": "...",
  "ath": "xVVHRsSiijn..."
}
```

Consumes:

- application/json:
  - credential_configuration_id (mandatory) String that uniquely identifies one of the keys in the name/value pairs stored in the credential_configurations_supported Credential Issuer metadata. Example = "university_example_sd_jwt")
  - proofs (optional) providing 1+ proof of possessions of the cryptographic key material to which the issued Credential instances will be bound to
  - credential_response_encryption (optional) providing information how to encrypt the Credential Response, if present.

```
{
    "credential_configuration_id": "university_example_sd_jwt",
    "proofs": {
        "jwt": [

"eyJ0eXAiOiJvcGVuaWQ0dmNpLXByb29mK2p3dCIsImFsZyI6IkVTMjU2IiwiandrIjp7Imt0eSI6IkVDIiwidXNlIjoic2lnIiwiY3J2IjoiUC0
yNTYiLCJraWQiOiJUZXN0LUtleS0wIiwieCI6Ii1QRXVmRjdnZmRta0dHVGQ5cG95Ykdqdmkz SUxpeWFSdDNHN0F5aWxOSmsiLCJ5IjoianMyRDN
HRzNsdEZEZGplbG1yYzVWby0tejFoTnJJT3VVdFFtTGs1RkozTSIsImlhdCI6MTc1NDU4NDE3OH19.
eyJhdWQiOiJodHRwOi8vbG9jYWxob3N0OjgwODAvb2lkNHZjaSIsIm5vbmNlIjoiYzljNzM3ZjMtZjM3Yi00MzMxLWEyM2MtMzEzZGM4MjFmYWM1
OjoyMDI1LTA4LTA3VDE2OjMwOjE5LjAyMTA0NTA3OFoiLCJpYXQiOjE3NTQ1ODQzMTl9.tq_ZvRbz7BoGOdMrPHwSw64gI-
wzqQ7sD4XfjdKIW3iL8rj7FX-s8Fpsed0C8qITUl3K_8UWRfeoLZwA85NKuA"
        ]
    },
    "credential_response_encryption": {
                "alg": "ECDH-ES",
        "enc": "A128GCM",
        "jwk": {"kty":"EC","crv":"P-256","kid":"transportEncKeyEC","x":"
DTaouFJpyVkLvfhoOvuTDR6_nmTt7YTvEHsHzK0Ingk","y":"vOipfo61Sy64XpneRyR5g6NCGXLv_Q7f3-kEDMT-G9U"
    }
}
```

- application/jwt
    - String Encrypted request body

9.3.3.2. Response formats:

- `Accept: application/json`
- `Accept: application/jwt` String

Response: `200 OK`

Errors: see Default Exception Handling

9.3.4. Deferred Endpoint

Issues a credential for a deferred transaction. Requires a valid bearer token and transaction details in the request body

- POST /oid4vci/api/deferred_credential

Response formats:

- `Accept: application/json` (default) NonceResponse

Response:

```
{
        "transaction_id": "$TRANSACTION_ID",
    "interval": 500
}
```

9.3.4.1. Response formats:

- `Accept: application/json` (default)
    - (optional / default) SWIYU-API-Version = 2
- `Accept: application/jwt` String

Response:
HTTP 202 Accepted (Waiting / Pending)
This status code **MUST** be returned if the credential cannot yet be issued. This occurs in two cases:

- **Initialization**: When the issuer cannot immediately respond to the initial request at the Credential Endpoint and instead returns a `transaction_id`.
- **Polling**: When the wallet queries the Deferred Credential Endpoint and issuance still takes time (`"issuance_pending"`). In this case, a `transaction_id` (and optionally an `interval`) is returned again.

HTTP 200 OK (Success)
This status code **MUST** be returned when the Deferred Credential Endpoint can successfully issue the credential (i.e. the process is complete and the credential is included in the response body).

As long as waiting is required, return **202**. Once the credential is delivered, return **200**.

Errors: see Default Exception Handling

9.4. Default Exception Handling

All endpoints use the application-wide default exception handler. Error responses follow this structure (aligned with your existing management API spec):

```
{
  "error": "ERROR_CODE (optional)",
  "error_description": "Human-readable error description",
  "detail": "Additional error details (optional)",
  "trace_id": "Trace identifier (optional)"
}
```

HTTP status codes:

- 200 OK: Successful response
- 400 Bad Request: Invalid parameters, malformed Accept header, etc.
- 401 Unauthorized: Missing or invalid authentication (if enforced)
- 403 Forbidden: Insufficient permissions (if enforced)
- 404 Not Found: Tenant not found or resource unavailable
- 500 Internal Server Error: Unexpected server errors

Examples:

- `404 Not Found`

```
{
  "error_description": "Not Found",
  "detail": "Tenant with ID 18fa7c77-9dd1-4e20-a147-fb1bec146085 not found"
}
```

10. Interface - Credential Metadata API Specification (IF-110)

10.1. Overview

The Credential Metadata API provides endpoints for retrieving metadata related to verifiable credentials managed by the issuer. It exposes:

- Verifiable Credential Type (VCT) metadata
- JSON Schema definitions for credential payloads
- Overlays Capture Architecture (OCA) metadata

Base URL: `/oid4vci` API Version: 1.2.4 Protocol: REST (JSON over HTTP/HTTPS)

10.2. Headers

10.2.1. Required/Recommended

- `Accept`: Determines the response format. See per-endpoint `produces` below.
  - For VCT and OCA endpoints: `application/json`
  - For JSON Schema endpoint: `application/schema+json`
- `Authorization`: `Bearer <token>` if required by deployment policy

10.2.2. Notes

- All endpoints are GET-only and do not require `Content-Type`.
- Responses are returned as raw strings from the service, but their media types are enforced per endpoint.

10.3. Endpoints

10.3.1. Get Credential Type Metadata (VCT)

Retrieve the Verifiable Credential Type metadata referenced by issuer metadata.

- GET `/oid4vci/vct/{metadataKey}`

Path parameters:

- `metadataKey` (string, required): Identifier for the VCT metadata (e.g., `my-vct-v01`)

Produces:

- `application/json`

Response: `200 OK` (example)

```
<content is in the mounted vct metadata file  in an unaltered form for SRI>
```

Errors: see Default Exception Handling

10.3.2. Get JSON Schema

Retrieve a JSON Schema used to validate the credential subject data.

- GET `/oid4vci/json-schema/{schemaKey}`

Path parameters:

- `schemaKey` (string, required): Identifier for the JSON schema (e.g., `university_example_sd_jwt`)

Produces:

- `application/schema+json`

Response: `200 OK` (example)

```
<content as string of the mounted json schema file in an unaltered form for SRI>
```

Errors: see Default Exception Handling

10.3.3. 3. Get Overlays Capture Architecture (OCA)

Retrieve OCA overlays for UI representation and localization of credential claims.

- GET `/oid4vci/oca/{ocaKey}`

Path parameters:

- `ocaKey` (string, required): Identifier for the OCA metadata (e.g., `university_example_sd_jwt`)

Produces:

- `application/json`

Response: `200 OK` (example)

```
<content as string of the mounted oca file in an unaltered form for SRI>
```

Errors: see Default Exception Handling

10.4. Content Types and Negotiation

- VCT metadata: `application/json`
- JSON Schema: `application/schema+json`
- OCA: `application/json`

Note: This controller enforces fixed `produces` media types per endpoint and does not support `application/jwt`.

10.5. Default Exception Handling

All endpoints use the application-wide default exception handler. Error responses follow this structure:

```
{
  "error": "ERROR_CODE (optional)",
  "error_description": "Human-readable error description",
  "detail": "Additional error details (optional)",
  "trace_id": "Trace identifier (optional)"
}
```

HTTP status codes:

- 200 OK: Successful response
- 400 Bad Request: Invalid parameters
- 401 Unauthorized: Missing or invalid authentication (if enforced)
- 403 Forbidden: Insufficient permissions (if enforced)
- 404 Not Found: Metadata key not found or resource unavailable
- 500 Internal Server Error: Unexpected server errors

Examples:

- `404 Not Found`

```
{
  "error_description": "Not Found",
  "detail": "VCT metadata with key 'my-vct-v01' not found"
}
```

11. Interface - Issuer Renewal API Specification (IF-201)

11.1. Data Models

11.1.1. RenewalResponseDto

Response model for a successful credential renewal operation.

```
 "metadata_credential_supported_id": ["myIssuerMetadataCredentialSupportedId"],
  "credential_subject_data": {
    "lastName": "Example",
    "firstName": "Edward"
  },
  "credential_metadata": {
    "vct#integrity": "sha256-0000000000000000000000000000000000000000000="
  },
  "credential_valid_until": "2010-01-01T19:23:24Z",
  "credential_valid_from": "2010-01-01T18:23:24Z",
  "status_lists": [
    "https://example-status-registry-uri/api/v1/statuslist/05d2e09f-21dc-4699-878f-89a8a2222c67.jwt"
  ],
  "configuration_override": {
    "issuer_did": "did:example:123456789abcdefghi",
    "verification_method": "did:example:123456789abcdefghi#keys-1",
    "key_id": "hsm-key-01",
    "key_pin": "****"
  }
}
```

Field Descriptions:

- metadata_credential_supported_id: List of IDs linking the offer to the issuer metadata (required).
- credential_subject_data: User data to be written in the verifiable credential. Can be a JSON object or a JWT (required).
- credential_metadata: Metadata for credential creation (optional).
- credential_valid_until: Expiry date/time for the credential (optional, ISO 8601 format).
- credential_valid_from: Start date/time for the credential (optional, ISO 8601 format).
- status_lists: List of URIs for status lists used with the credential (optional).
- configuration_override: Optional overrides for cryptographic/DID settings (all fields optional).

11.2. Endpoints

11.2.1. Renew Credential

Renew an existing credential for a subject.

Endpoint:

POST /api/issuer/renewal

Request Body:

```
{
  "credential_id": "string",
  "subject_id": "string",
  "credential_subject_data": {
    "lastName": "Example",
    "firstName": "Edward"
  },
  "configuration_override": {
    "issuer_did": "did:example:123456789abcdefghi",
    "verification_method": "did:example:123456789abcdefghi#keys-1",
    "key_id": "hsm-key-01",
    "key_pin": "****"
  }
}
```

Returns a RenewalResponseDto as described above.

11.3. Example

11.3.1.
Example 1: Renew a Credential

Request:
POST /api/issuer/renewal
Content-Type: application/json

```
{
  "credential_id": "123e4567-e89b-12d3-a456-426614174000",
  "subject_id": "user-001",
  "credential_subject_data": {
    "lastName": "Example",
    "firstName": "Edward"
  }
}
```

Response:

```
{
  "metadata_credential_supported_id": ["myIssuerMetadataCredentialSupportedId"],
  "credential_subject_data": {
    "lastName": "Example",
    "firstName": "Edward"
  },
  "credential_metadata": {
    "vct#integrity": "sha256-0000000000000000000000000000000000000000000="
  },
  "credential_valid_until": "2010-01-01T19:23:24Z",
  "credential_valid_from": "2010-01-01T18:23:24Z",
  "status_lists": [
    "https://example-status-registry-uri/api/v1/statuslist/05d2e09f-21dc-4699-878f-89a8a2222c67.jwt"
  ],
  "configuration_override": null
}
```

12. Interface - Webhook Callback API Specification (IF-202)

12.1. Data Models

12.1.1. WebhookCallbackDto

Callback model for events occurring in the system, sent to external systems.

```
{
  "subject_id": "123e4567-e89b-12d3-a456-426614174000",
  "event_type": "VC_STATUS_CHANGED",
  "event": "CREDENTIAL_ISSUED",
  "event_description": "Credential was successfully issued.",
  "timestamp": "2024-06-13T10:15:30Z"
}
```

Field Descriptions:
subject_id: UUID of the subject related to the event (required).
event_type: Type of event. Possible values: VC_STATUS_CHANGED, VC_DEFERRED, ISSUANCE_ERROR (required).
event: Name of the specific event (required).
event_description: Description of the event (optional). see **CredentialOfferStatusType**
timestamp: Time of the event in ISO 8601 format (required).

CredentialOfferStatusType
Represents the possible states of a credential offer.

Possible values:

- INIT: Initial state.
- OFFERED: Credential has been offered.
- CANCELLED: Offer was cancelled.
- IN_PROGRESS: Claiming is in progress.
- DEFERRED: Deferred flow is active.
- READY: Credential is ready to be issued.
- ISSUED: Credential has been issued.
- REQUESTED: Renewal flow, credential has been requested.
- EXPIRED: Credential offer has expired.

12.2. Functionality

The system collects callback events and periodically sends them via HTTP POST to the configured callback URL. Events are deleted after successful delivery. Failed deliveries are retried on the next run (at-least-once delivery).

12.3. Configuration

The target URL and optional authentication headers are set in the application configuration *(webhook.callbackUri, webhook.apiKeyHeader, webhook.apiKeyValue).*

12.4. Example

Request:
POST <callbackUri>
Content-Type: application/json

```
{
  "subject_id": "123e4567-e89b-12d3-a456-426614174000",
  "event_type": "VC_STATUS_CHANGED",
  "event": "EXPIRED",
  "event_description": "Credential status changed to EXPIRED.",
  "timestamp": "2024-06-13T10:15:30Z"
}
```

Response:
HTTP 204 No Content (on success)
Failed deliveries are automatically retried.

13. Runtime - Issuance Process - Complete

13.1. Overview of oid4vci flow - Pre-Authorized Code Flow

The Pre-Authorized Code Flow is a specific process within OpenID for Verifiable Credential Issuance (OID4VCI).

It is designed for situations where the Credential Issuer (the entity issuing digital credentials) has already prepared the issuance of a digital credential and has obtained the End-User's consent outside of the standard OAuth process.

Description how the Pre-Authorized Code Flow works:

1. Preparation by the Credential Issuer: The Credential Issuer first gathers all necessary information and obtains the End-User's consent for issuing the credential through its own internal business process.
   This preparation phase is outside the scope of the technical specification.

2. Credential Offer: The Credential Issuer then creates a Credential Offer that includes a Pre-Authorized Code. This offer is sent to the End-User's Wallet, as a scannable QR code or a URI.
   The offer also contains the Credential Issuer's URL and details about the specific credentials being offered.

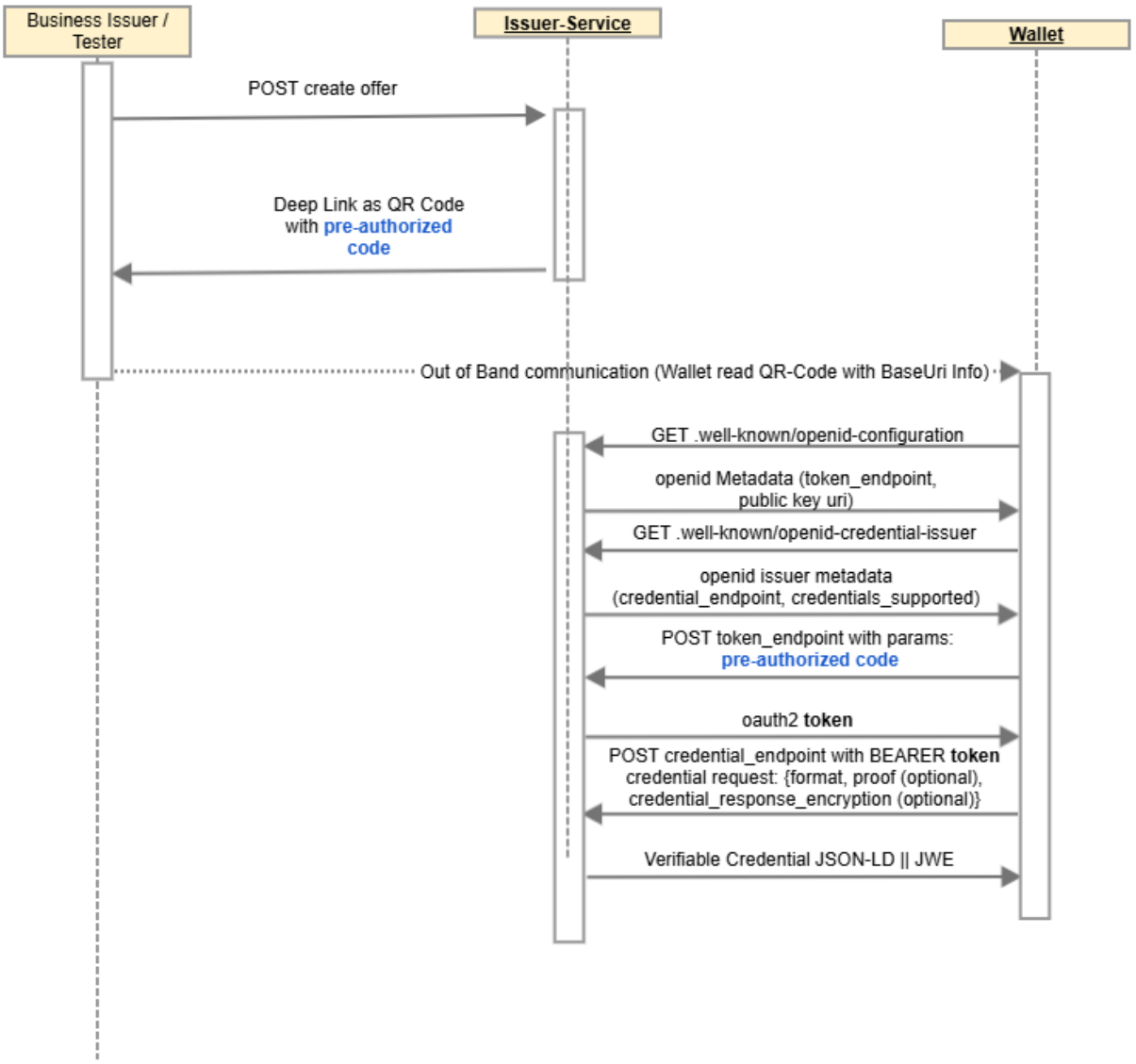
3. Wallet Fetches Metadata: The End-User's Wallet uses the Credential Issuer's URL from the offer to retrieve the Issuer's metadata.
   This metadata informs the Wallet about the types and formats of credentials the Issuer supports, and provides the URLs for the necessary endpoints, such as the Token Endpoint and Credential Endpoint.

4. Token Request (with Pre-Authorized Code): The Wallet then sends the Pre-Authorized Code directly to the Token Endpoint of the Credential Issuer.

5. Token Response (Access Token): If the Pre-Authorized Code is successfully validated, the Token Endpoint returns an Access Token to the Wallet .

6. Credential Request: The Wallet then uses the acquired Access Token to send a Credential Request to the Issuer's Credential Endpoint. This request typically includes a "proof of possession" of the private key to which the digital credential will be cryptographically bound.

7. Credential Response: The Credential Issuer verifies the request and sends the digital credential(s) back to the Wallet. If the issuance cannot be immediate, the Issuer may return a webhook-uri instead, which the Wallet can use later at the Deferred Credential Endpoint to retrieve the credential.



13.2. Detailed View

This sequence diagram describes the detailed end-to-end flow for issuing a verifiable credential using the generic issuer service. It shows the interactions between the four main actors:

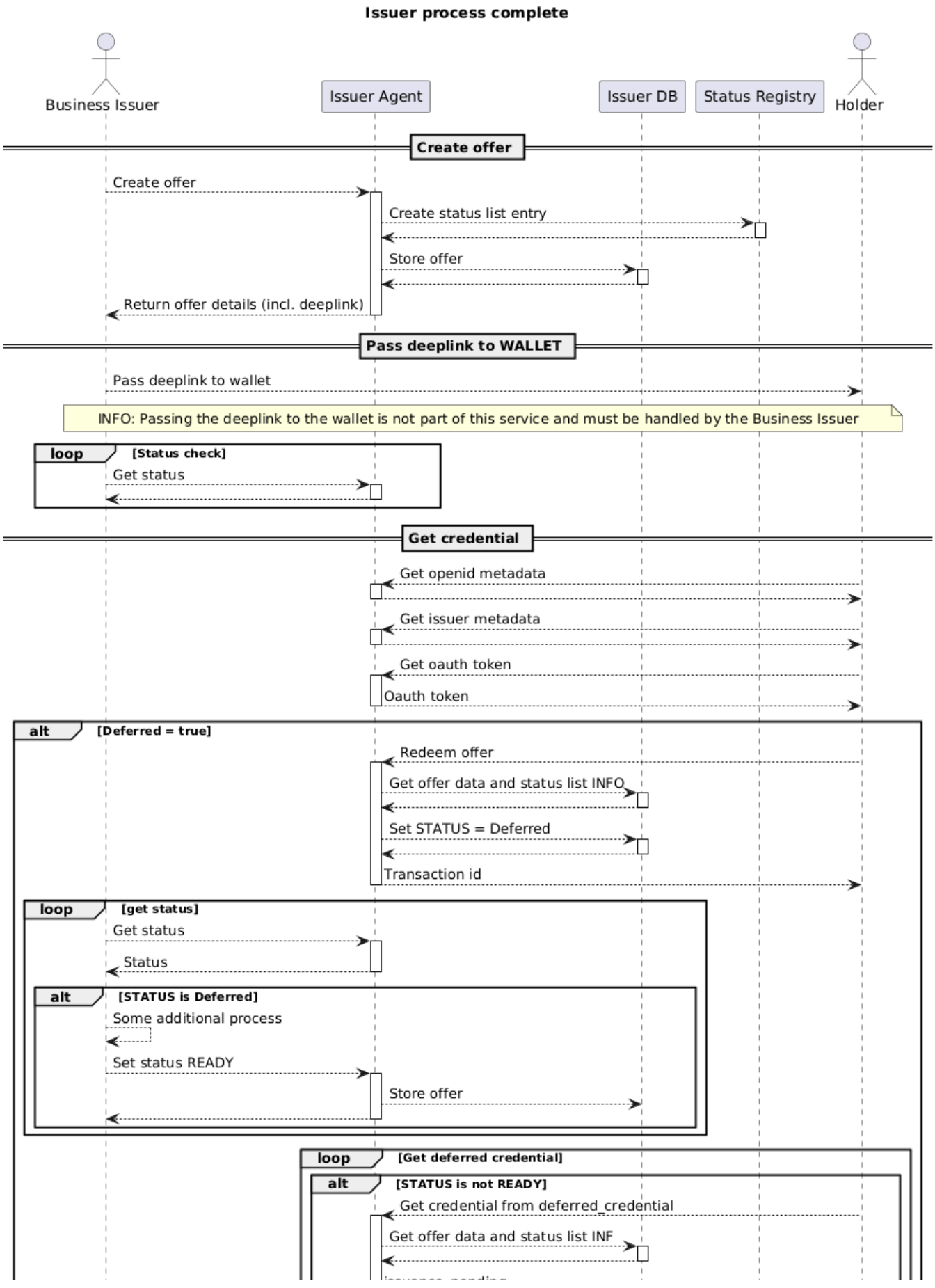
- Business Issuer
- Issuer Agent
- Status Registry
- Holder's Wallet.

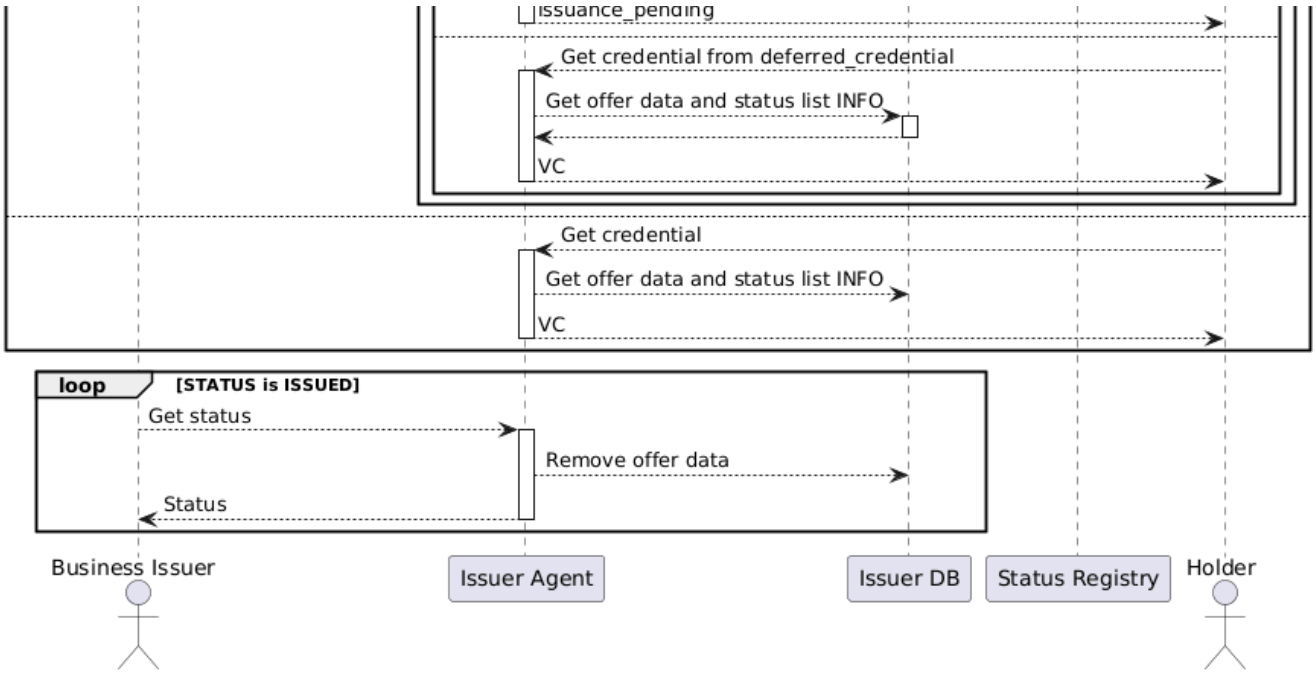
Key steps:

1. The Business Issuer creates an offer via the Issuer Agent, which stores offer data and status in the database and status registry.
2. The Business Issuer passes a deeplink to the Wallet (outside the service scope).
3. The Wallet interacts with the Issuer Agent to fetch metadata, obtain an OAuth token, and redeem the offer.
4. If the credential issuance is deferred, the status is updated in the database, and the Business Issuer must set the status to READY before the Wallet can retrieve the credential.
5. If not deferred, the Wallet can directly obtain the credential.

6. After issuance, the Business Issuer can check the status, and the Issuer Agent removes offer data when the credential is issued.

The diagram covers both immediate and deferred issuance flows, including status checks and transitions.


Issuer process complete
Business Issuer
Issuer Agent
Issuer DB
Status Registry
Holder
Create offer
Create offer
Create status list entry
Store offer
Return offer details (incl. deeplink)
Pass deeplink to WALLET
Pass deeplink to wallet
INFO: Passing the deeplink to the wallet is not part of this service and must be handled by the Business Issuer
loop
[Status check]
Get status
Get credential
Get openid metadata
Get issuer metadata
Get oauth token
Oauth token
alt
[Deferred = true]
Redeem offer
Get offer data and status list INFO
Set STATUS = Deferred
Transaction id
loop
[get status]
Get status
Status
alt
[STATUS is Deferred]
Some additional process
Set status READY
Store offer
loop
[Get deferred credential]
alt
[STATUS is not READY]
Get credential from deferred_credential
Get offer data and status list INF



13.3. Deferred Mode

The "Deferred Mode" (also known as "Deferred Issuance") is a feature in the OpenID for Verifiable Credential Issuance (OpenID4CI) specification. It is used when a credential issuer cannot issue a requested Verifiable Credential (VC) immediately.

In such cases, Deferred Mode allows the issuer to delay the issuance. This can be necessary, for background checks or other time-consuming steps.

Deferred Credentials allows the Issuer to return a `transaction_id` instead of a credential in the issuing flow credential response. The URL for the wallet to poll is provided through the issuer metadata `deferred_credential_endpoint`.

To maintain flexibility (needed at least in the demo environments to enable testing of the wallet), the Business Issuer must be able to control credential deferral in an offer. To provide the desired flow a boolean is provded in the `credential_metadata` during offer creation. For context: this is the same json object as used for providing `vct#integrity`

Credential Offer Metadata

```
"credential_metadata": {
        "deferred": true
}
```

For this purpose 2 new states are introduced: **DEFERRED** & **READY**

13.3.1. Flow with deferral

Changed from the regular non-deferred flow, the holder's call to the `credential_endpoint` receives a `transaction_id` instead of the credential directly. The holder only knows that it must start polling the `deferred_credential_endpoint` as it received `transaction_id` instead of credential in the `credential_endpoint` response.

credential_endpoint response

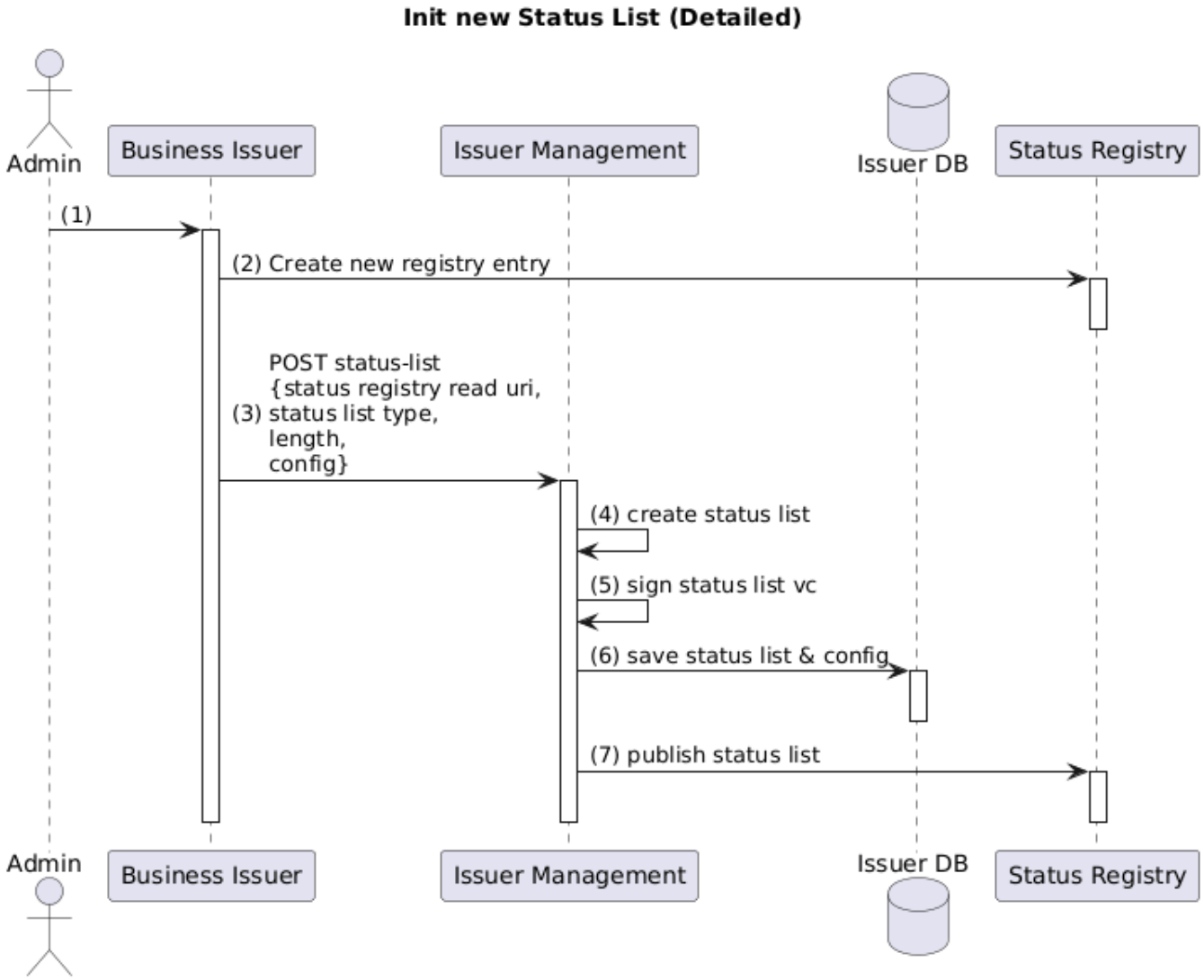
```
{
  "transaction_id": "5e7f4abe-5f0f-46a5-a43f-abe7cb465849",
  "c_nonce": "78fa2903-95e1-4b50-a5ca-25886f97c3c9",
  "c_nonce_expires_in": 86400
}
```

We should use the optional `c_nonce` to keep an option to create a proof that subsequent calls are made by the same holder instance. We use a (fresh) UUID as `c_nonce` value, replacing the one we before used for proof.

The issuer must save the public key provided in the proof in the `credential_metadata` for the business issuer to use in key attestation checks.

14. Runtime - Initializing a new Status List

To use a status list, it's required first to create a slot on the status registry. Once this has been created the status list needs to be initialized. During that initialization process, the type of used status list, it's size and other status list type specific configuration is selected through the JSON request body parameters. Once created locally a Status List VC is created and published on the status registry.



15. Runtime - Update Status List

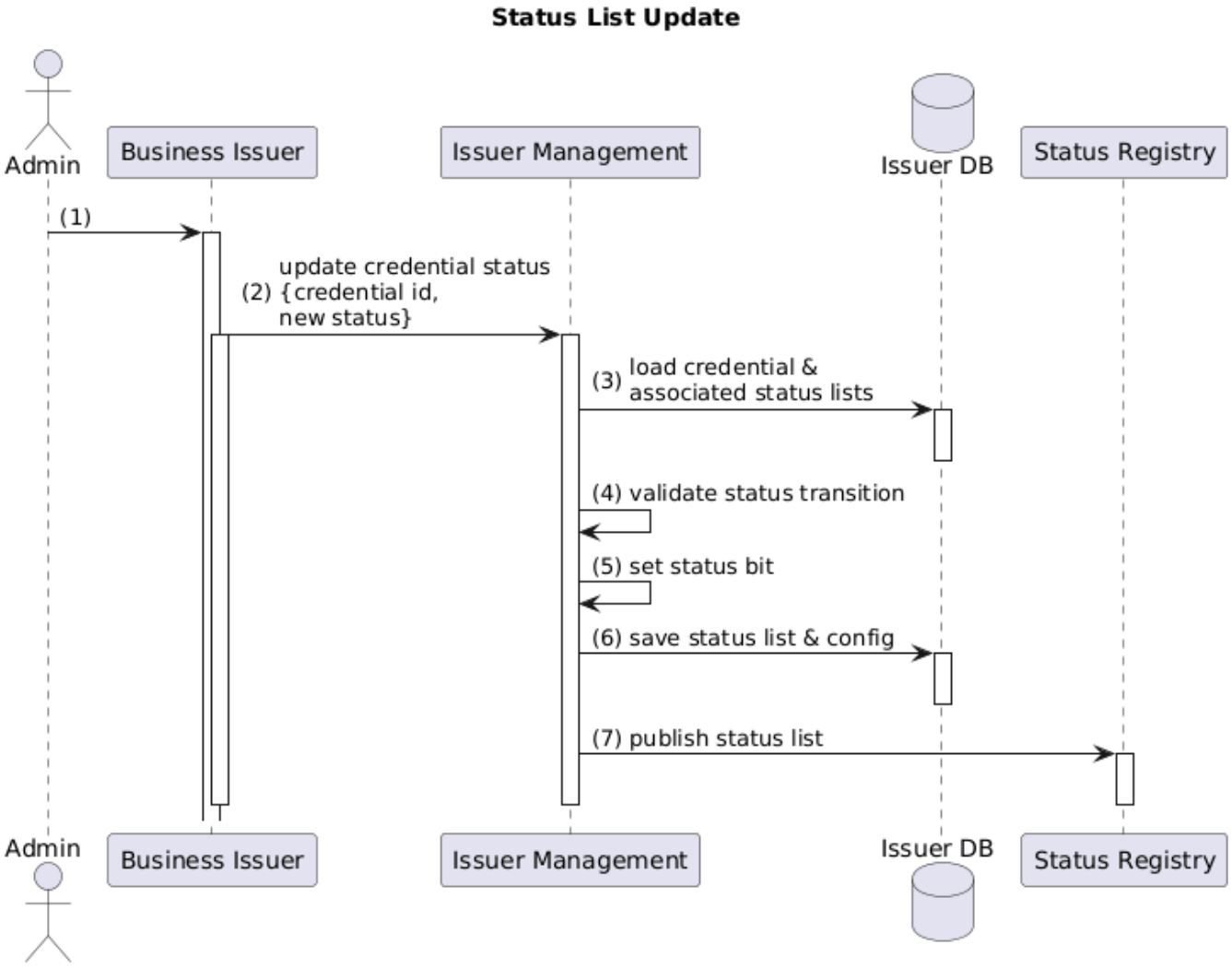
Status Lists are updated every time a credential status is updated. The bit corresponding to the credential's status is set, a new VC created, signed and published to the status registry.

This sequence diagram describes the process of updating the status of a credential in the system:

1. The Admin initiates the process by interacting with the Business Issuer.
2. The Business Issuer sends a request to the Issuer Management component to update the status of a credential, providing the credential ID and the new status.
3. Issuer Management loads the credential and its associated status lists from the Issuer DB.
4. Issuer Management validates whether the status transition is allowed.

5. Issuer Management updates the status bit for the credential.
6. The updated status list and configuration are saved back to the Issuer DB.
7. Issuer Management publishes the updated status list to the Status Registry.

   The process completes, and all participants deactivate in reverse order.



16. Runtime - Renewal Flow - Batch Issuance

16.1. Abstract

This concept describes the implementation of the Renewal Flow for the Issuance of Verifiable Credentials (VCs). The primary context of this procedure is (Batch-) Issuance with Renewal Key, which aims to prevent the Traceability (Rückverfolgbarkeit) of individuals within the framework of electronic identity (e-ID) usage

16.2. Core Objectives and Methodology

To ensure Unlinkability (Nicht-Verknüpfbarkeit), the Holder (Wallet) is encouraged to use Single-Use Credentials. Once a set of these Credentials is exhausted, the Renewal Flow enables the Holder to automatically request a new set of Credentials (Batch) from the Issuer.

The Authentication of the Holder with the Issuer is carried out using a dedicated Renewal Key Pair (sk renew / pk renew) and an associated JWT renew. This Key Pair serves exclusively for communication with the Issuer to obtain new VCs, and is not disclosed otherwise.

16.3. High-Level Technical Flow (OID4VCI and OAuth 2.0)

The Renewal Flow is based on the standards OpenID for Verifiable Credential Issuance (OID4VCI) and OAuth 2.0. The process utilizes the OAuth 2.0 Grant Type grant_type=refresh_token. The Wallet performs the following main steps:

Refreshing an Access Token with OAuth 2.0 and Demonstrating Proof of Possession

1. Nonce Request: The Wallet can request a fresh c_nonce value from the Nonce Endpoint of the Issuer to ensure the Freshness (Aktualität) of the Proofs of Possession (PoP) and prevent Replay-Attacks.
2. Token Exchange: The Wallet sends the refresh_token to the /token-Endpoint, along with a DPoP-Proof (Demonstrating Proof of Possession, RFC 9449). Upon success, the Wallet receives a new access_token.

Credential Request (Renewal Flow)

3. Credential Request: The Wallet sends a Credential Request (Batch Issuance) to the /credential-Endpoint, authorized by the new Access Token. This request includes Proofs of Possession (Proofs) for the cryptographic keys of the newly requested Credentials, typically provided as JWTs or Key Attestations.

16.4. Fundamentals and Architectural Components

16.4.1. Batch Issuance and Unlinkability

Batch Issuance is a core architectural concept that addresses the challenge of ensuring Unlinkability in the usage of electronic identity (e-ID) and preventing the Traceability of individuals. The system design enables the issuance of a batch of one or more Verifiable Credentials (VCs) in a single request to the Credential Endpoint.

Core Mechanisms:

- **Multiple VC Issuance (Batch-Issuance):** To enhance privacy and minimize the risk of traceability, the issuer issues multiple VCs in a single batch. This allows the holder (wallet) to use different VCs for different presentations or verifiers, depending on its own policy, which contributes to achieving unlinkability. Once a batch is exhausted, the wallet automatically requests a new batch from the issuer via the renewal flow.
- **Cryptographic Diversity:** To ensure unlinkability, the credentials issued within the same batch MUST share the same credential format and credential dataset, but SHOULD contain different cryptographic data. This means that each credential should be bound to different cryptographic keys.
- **Renewal Key:** For the authentication of the Holder with the Issuer to request the new natch, a dedicated Renewal Key Pair (sk renew / pk renew) is utilized. This specific key pair is exclusively shared between the Holder and the Issuer and is only used for obtaining new VCs, thus ensuring that the security properties remain intact over time.

16.4.2. The Renewal Key

The Renewal Key is a dedicated cryptographic key pair that enables the re-authentication of the Holder with the Credential Issuer in a secure manner using the OAuth 2.0 refresh token. Using a key binding the refresh_token can be used over a longer time span than deemed secure in regular OAuth 2.0. Therefore removing the need to re-authenticate the end-user through the identity verification process once the active tokens have expired.

16.4.2.1. Function and Structure

- **Key Pair Generation:** The Holder (Wallet) generates the dedicated key Pair (sk renew / pk renew) during the initial issuance of the e-ID. This key pair is an ECDSA Key Pair (NIST p-256).
- **Dedicated Use:** The Renewal Key serves exclusively for communication with the Issuer. It is NOT used for other purposes or otherwise disclosed.
- **Authentication:** To request the renewal batch, the Wallet authenticates with the issuer by sending the JWT renew together with a Proof of Possession (PoP) of the associated private Renewal Key (sk renew)

16.4.2.2. Security Aspects and Lifecycle of the Renewal Key

- **Unlinkability:** Since the Renewal Key is used only for communication with the Issuer and NOT for the presentation of VCs, the security chain remains intact. Every newly issued credential remains securely bound to the Holder.
- **Replay Protection:** The renewal protocol MUST ensure that replay attacks are not possible, for instance, by employing a challenge (Nonce) with every Renewal Request.
- **Lifetime:** The Renewal Key is never renewed. Its validity defines the maximum lifetime of the batch-issued credential before a complete re-verification and re-issuance of the e-ID is required.
- **Hardware Binding:** During the renewal process, the Issuer can utilize Key Attestation (or, in the case of iOS, app attestation) to verify that both the Renewal Key and the key pairs of the newly requested VCs are protected on the same secure secure hardware element (Secure Enclave).

16.4.2.3. Proof of Possession (PoP) of the Renewal Key

- **PoP Mechanism:** The Proof of Possession (PoP) is a cryptographic mechanism through which the Holder (Wallet) proves control over the private Renewal Key (sk renew) to the Issuer. This is essential for identifying and authenticating the Holder and authorizing the request for new verifiable credentials (VCs).
- **Standard:** The PoP is established by creating a JWT using sk_renew according to OAuth 2.0 "DPoP" (Demonstrating Proof of Possession, RFC 9449).
- **Flow:** The Wallet transmits the DPoP proof along with the refresh_token to the /token-Endpoint to obtain a new access_token.
- **Integrity and Freshness:** To prevent replay attacks, the Wallet must request a fresh c_nonce value from the Nonce Endpoint of the issuer to guarantee the Freshness (Aktualität) of the Proof of Possession.

- **Binding:** The transmitted JWT renew SHOULD be sent to the Issuer together with the PoP of the sk renew to verify the validity of the e-ID and ensure the Authenticity of the exchanged data.

16.5. Technical Standards and Protocols

16.5.1. OID4VCI (OpenID for Verifiable Credential Issuance)

Concept	Standard	Relevant Chapters / Sections	
Batch Credential Issuance	OID4VCI (1.0)	**Defines the issuance of multiple credentials in a single request.** It is explicitly stated that the credentials within a given batch should share the **same format and dataset** but contain **different cryptographic data** to achieve unlinkability (e.g., binding to different cryptographic keys). Note that, for example, the dataset can change *between* batches. Further implementation considerations can be found in section 14.6. Batch Issuing Credentials.	OpenID for Verifiable Credential Issuance 1.0 - 3.3.2. Batch Credential Issuance
Batch Issuance Meta Data	OID4VCI (1.0)	Support for batch issuance is defined in the issuer's optional metadata object **`batch_credential_issuance`** in which the size of the batch being issued is defined in **`batch_size`**. From the perspective of the generic issuer, the batch size can be 1. In this case, no **`batch_credential_issuance`** is sent.	OpenID for Verifiable Credential Issuance 1.0 - 12.2.4. Credential Issuer Metadata Parameters

16.5.2. OAuth 2.0 Grant Type: Refresh Token

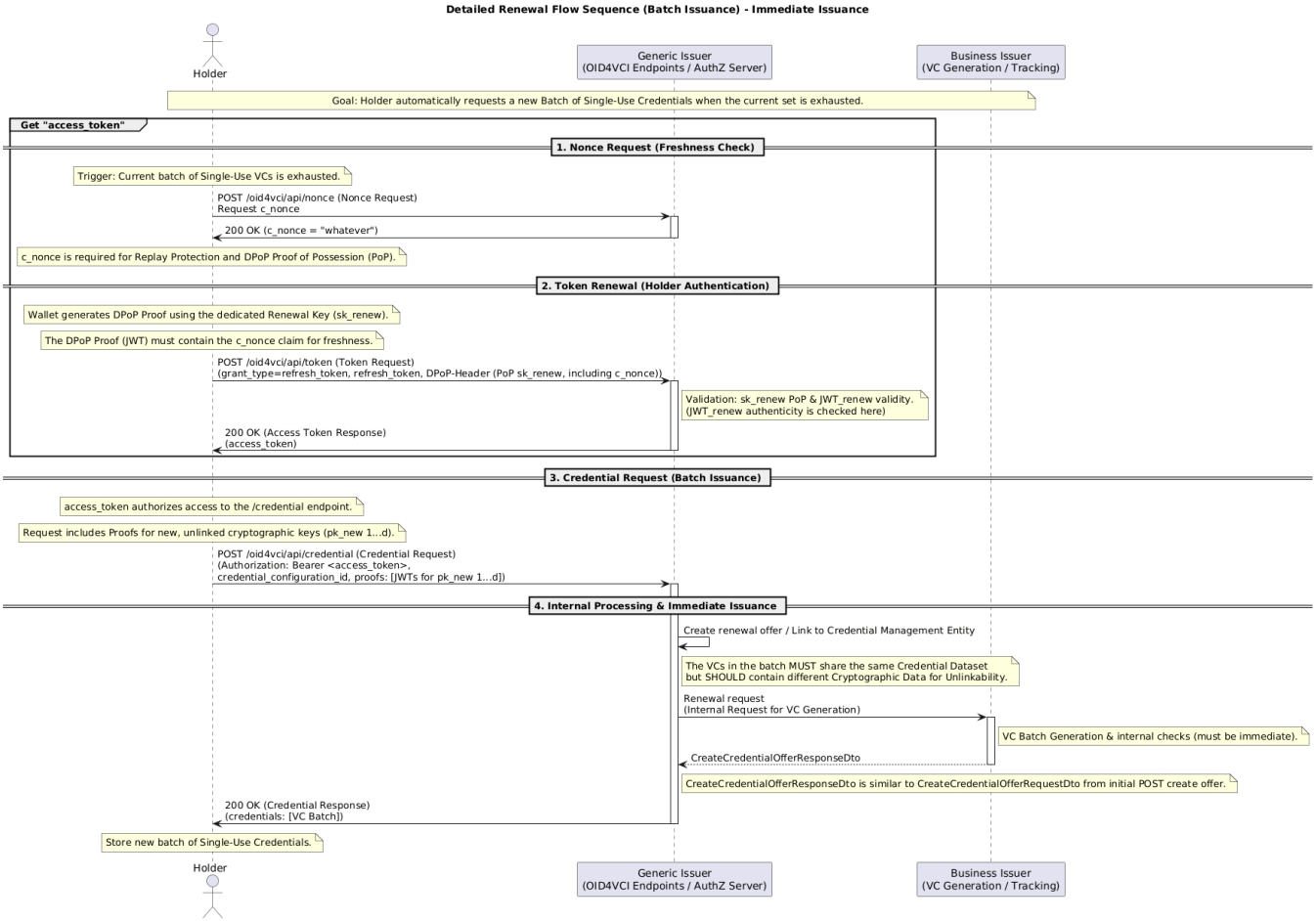
Concept	Standard	Relevant Chapters / Sections	
Refresh Token	RFC 6749 (OAuth 2.0)	**1.5. Refresh Token:** Refresh tokens are credentials used to obtain new access tokens when the current token becomes invalid or expires. They are intended solely for use with authorization servers (token endpoint).	RFC 6749 - The OAuth 2.0 Authorization Framework - 1.5. Refresh Token
Refreshing an Access Token	RFC 6749 (OAuth 2.0)	**6. Refreshing an Access Token:** Describes the flow in which the client (with `grant_type=refresh_token`) and the refresh token are sent to the token endpoint to obtain a new access token.	RFC 6749 - The OAuth 2.0 Authorization Framework - 6. Refreshing an Access Token

community/tech-concepts/e-id-key-management-batch-issuance-and-renewal.md at main · swiyu-admin-ch/community

Renewal Idea - eID Team Omni - Confluence

16.6. Technical Implementation

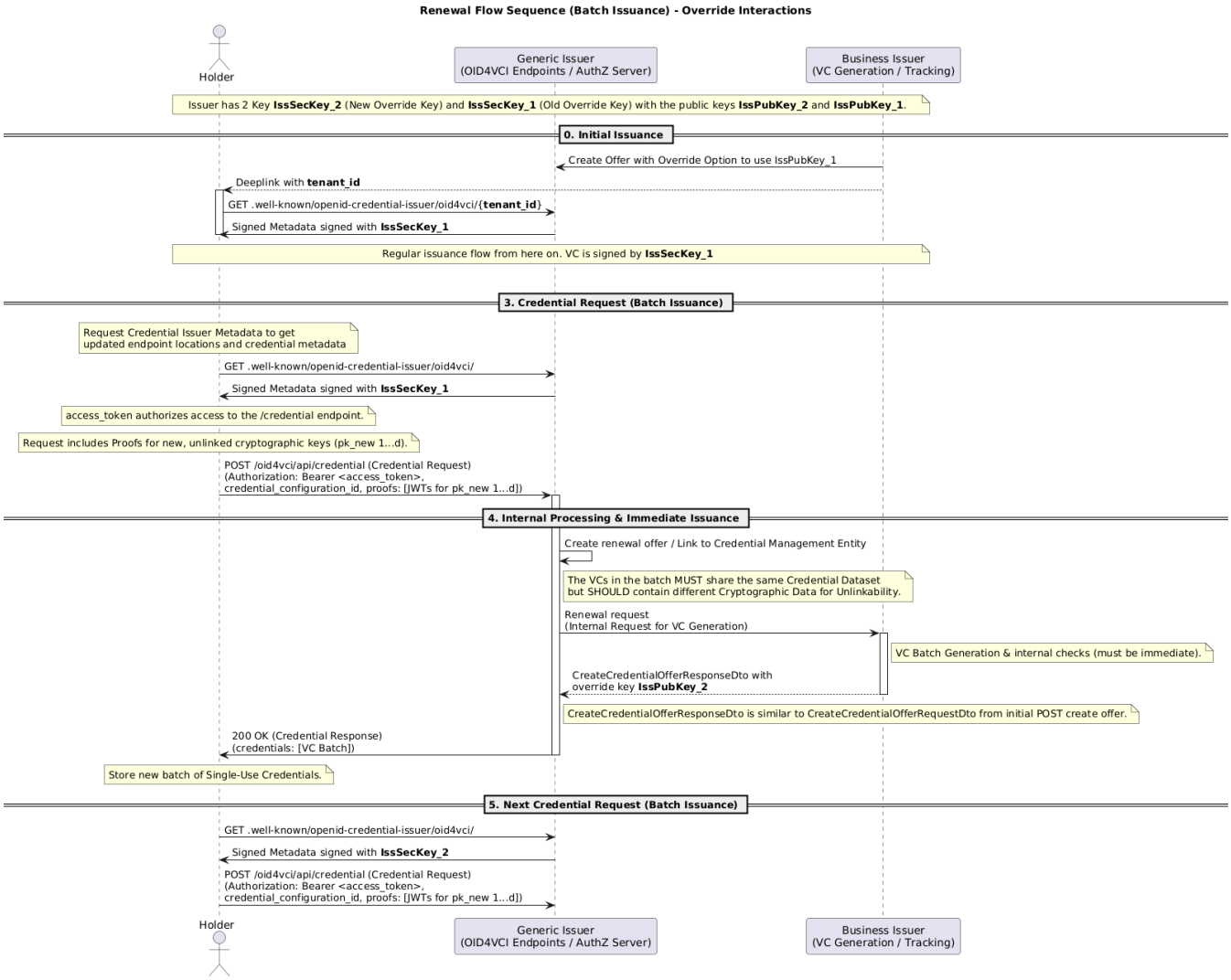
16.6.1. Detailed Process of the Renewal Flow



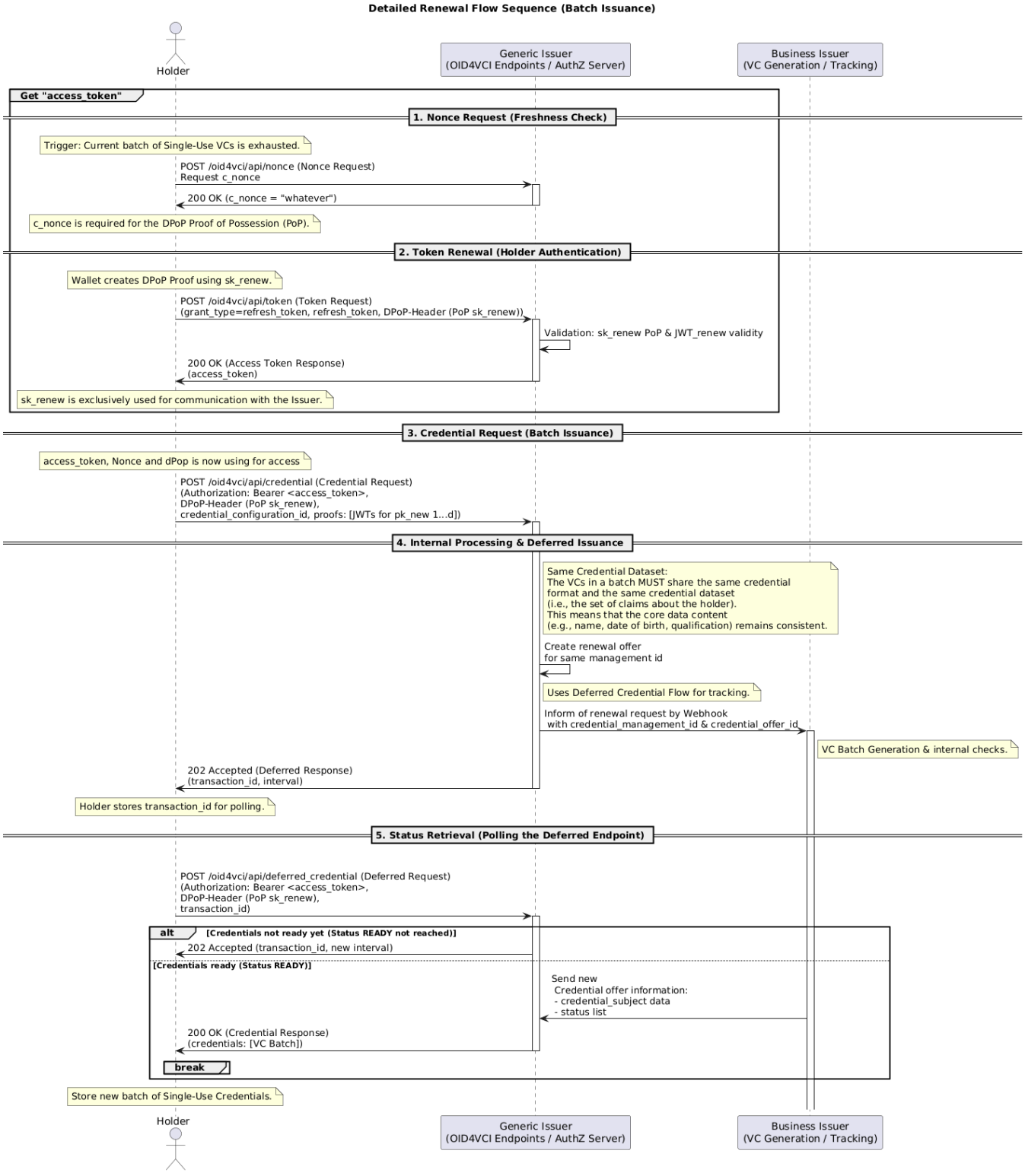
16.6.2. Renewal Flow with changing Issuer Keys

Issuers can choose what signing keys are used with signing metadata and credentials. This is done by using the override parameter.

Issuers can exchange their used signing keys using the override parameters by providing new override keys in the next renewal flow. This can be also be used if not using override keys at first.



16.6.3. Detailed Process of the Renewal Flow with deferred flow (not supported now)



Currently, the generic issuer does not offer a deferred flow for the renewal flow.

Decision

In discussions with the Wallet team (Communis) and the Issuer (Donum), it became clear that the deferred flow is not the right solution in this case, because the wallet wants to receive the response directly without having to poll for it. Therefore, the deferred flow will **not** be used in the renewal flow.

Issue: How does the generic issuer know which VC and which data need to be issued?

This data is only available during the initial issuance, for example via a QR code.

To prevent traceability of individuals, the newly issued credentials (batch issuance) must be bound to new, unlinked cryptographic keys (pk i+1 through pk i+d). Since these credentials are intentionally single-use and technically decoupled, the issuer lacks an automatic link between the original credential offer and all subsequent batches.

The Generic Issuer knows the presented renewal key. From this key it must get the concerned credentialManagementId.

The Business Issuer then receives the credentialManagementId from Generic Issuer. With this information, the business issuer knows what data to use for the new VC(s).

It is IMHO only a renewal question, unrelated to deferred or not.

16.6.4. Restoring the Logical Chain (Status Tracking)

To restore the necessary logical chain for the renewal flow, a "management entity" is introduced. This table records which credential offers have been created to renew the original offer.

When the issuer receives a Renewal Request and VC refresh is enabled, the generic issuer opens a new credential offer that inherits from the original credential offer.

These linked credential offers are connected through a **CREDENTIAL MANAGEMENT** table. This "credential management" serves as an abstraction layer, allowing all actions performed by the issuer (e.g., providing the VC data) to be tracked through it.

16.6.4.1. Linking Associated credential offers via a Management Entity

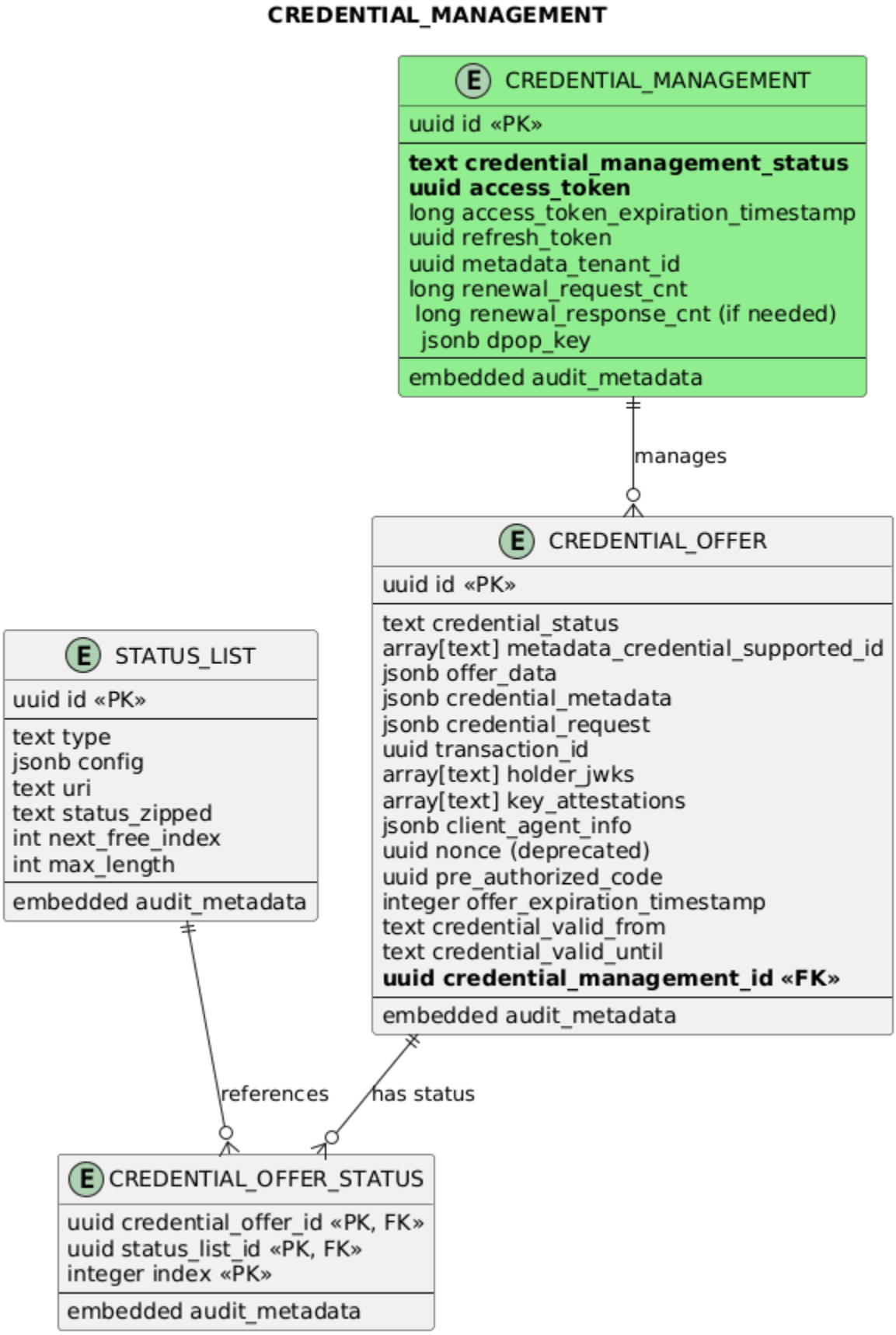
All actions performed by the business issuer are routed through the management entity, abstracting the refresh functionality details from the issuer. The existing API, including issuer management endpoints, remains unchanged.

Offers that use a **transaction_id** for deferred processing are already implemented. The business issuer can determine whether the same message is received multiple times or whether data needs to be provided repeatedly. This approach also allows more granular control over deferred credentials if needed.

Migration of DB Schema

For the DB schema migration, each existing record in **credential_offer** is assigned a new management entity to reference.

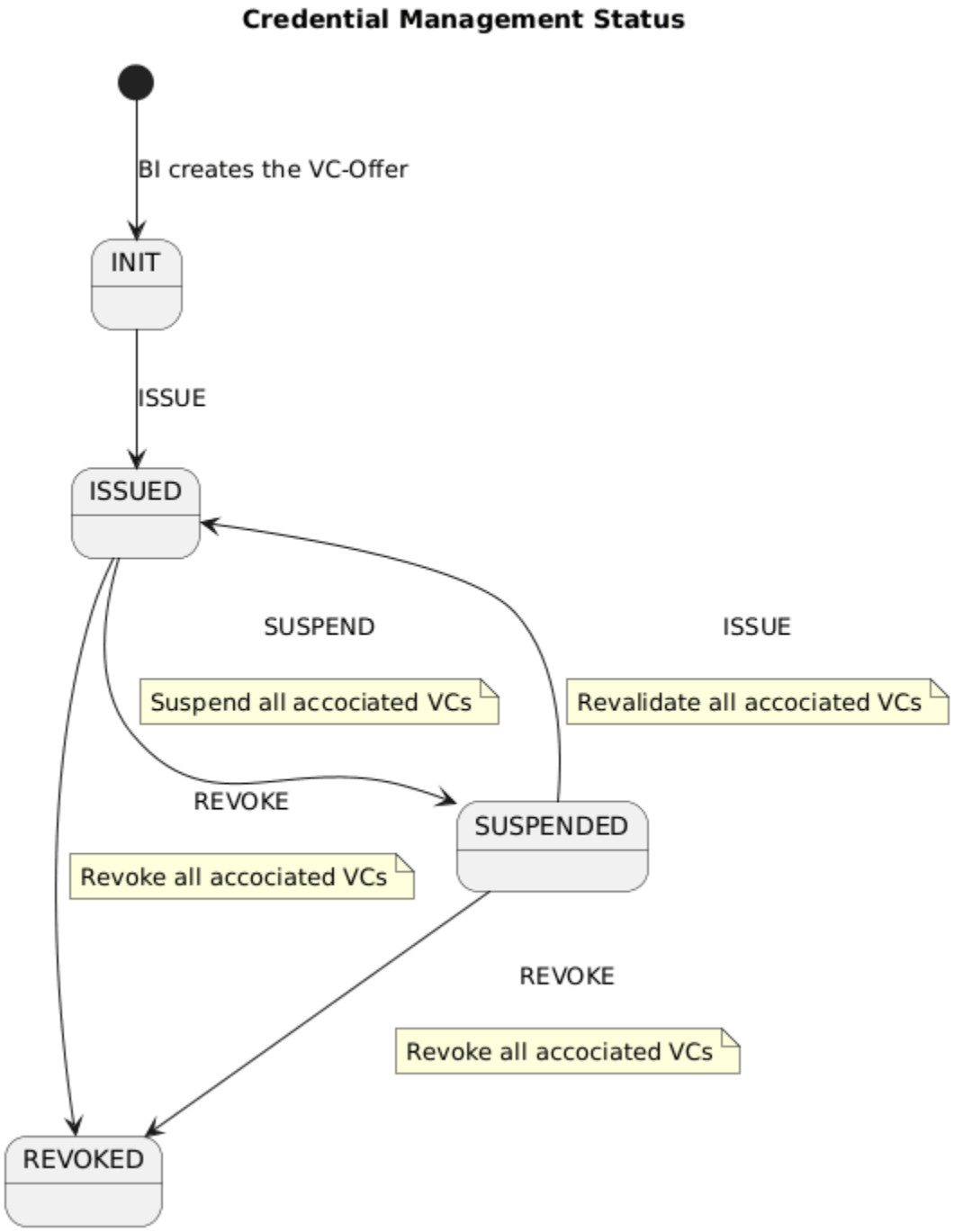
To avoid requiring a DB schema migration on the side of the business issuer, the newly created management entities are assigned the same UUID as their corresponding **credential_offer** records.



16.6.4.2. Status of the Issuance/Renewal Flow

6.1.5 Issuer - State-Event-Diagramm

16.7. Credential Management Status



The state machine for the **Credential Management** represents the overall state of all associated offers and controls the **SUSPENDED** and **REVOKED** states.
The **REVOKED** state must correlate with the corresponding state in the status lists.

16.7.1.

Renewal in Suspended / Revoked

Renewal in Suspended / Revoked

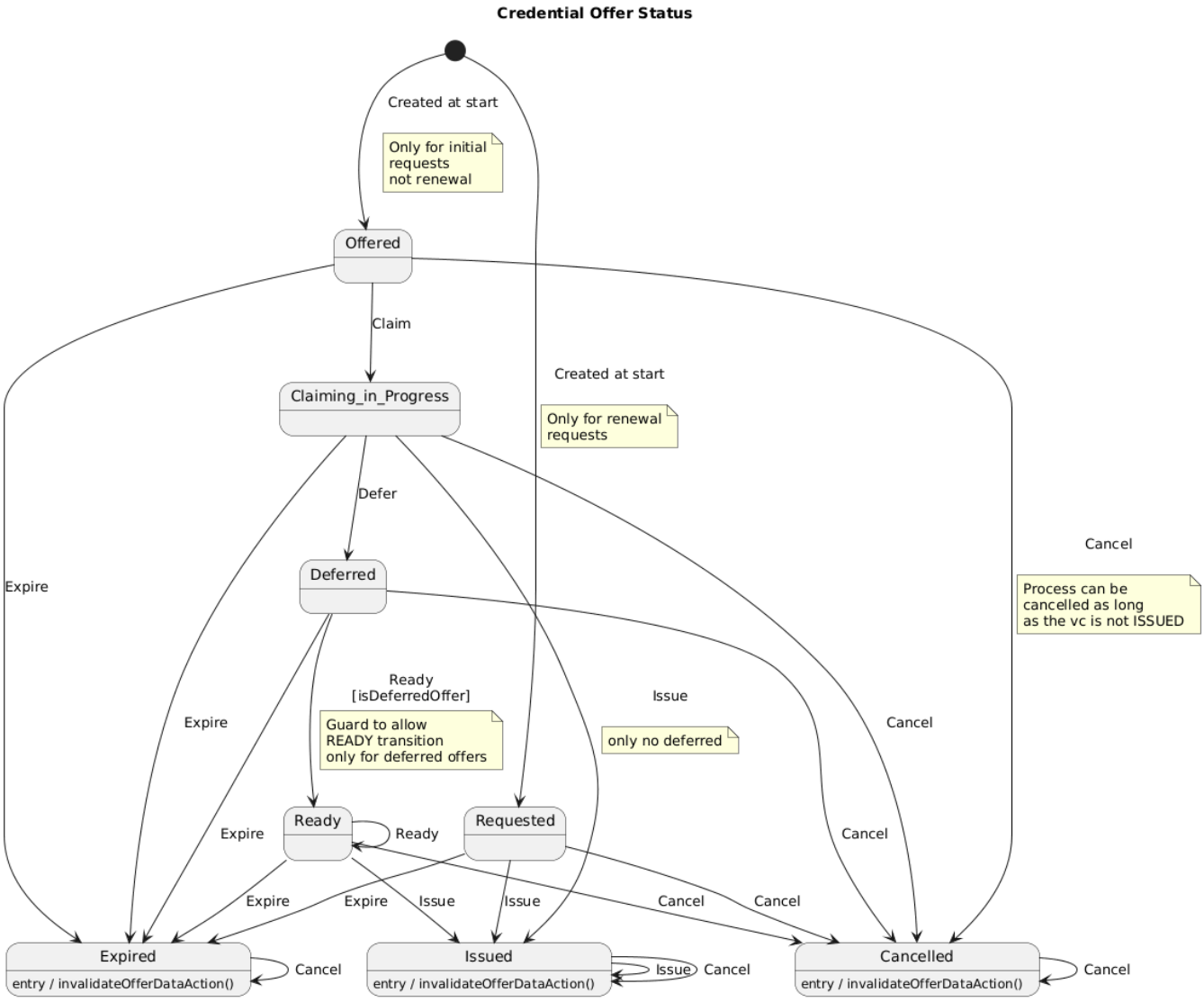
If the Management Entity is in the state **Suspended** or **Revoked**, renewal is currently not permitted and the renewal request is aborted.

The reason is that new status list entries are currently always initialized with `0`, which corresponds to **"Issued"**. Setting the status to **Suspended** or **Revoked** can take some time due to the database structure.

As a result, we cannot guarantee that newly created VCs within a batch will receive the status **Revoked** or **Suspended** before they are presented for the first time.

Due to this decision, **Suspended** (and **Revoked**) VCs may potentially be trackable if they are presented frequently.

16.8. Credential Offer Status



A new state called REQUESTED will be introduced.
This state indicates that a new offer has been requested via the credential renewal flow.
The credential offer remains in this state until the Business Issuer provides the necessary information and status lists, at which point the state transitions to READY.

16.8.1. Renewal of Revoked and Suspended Credentials

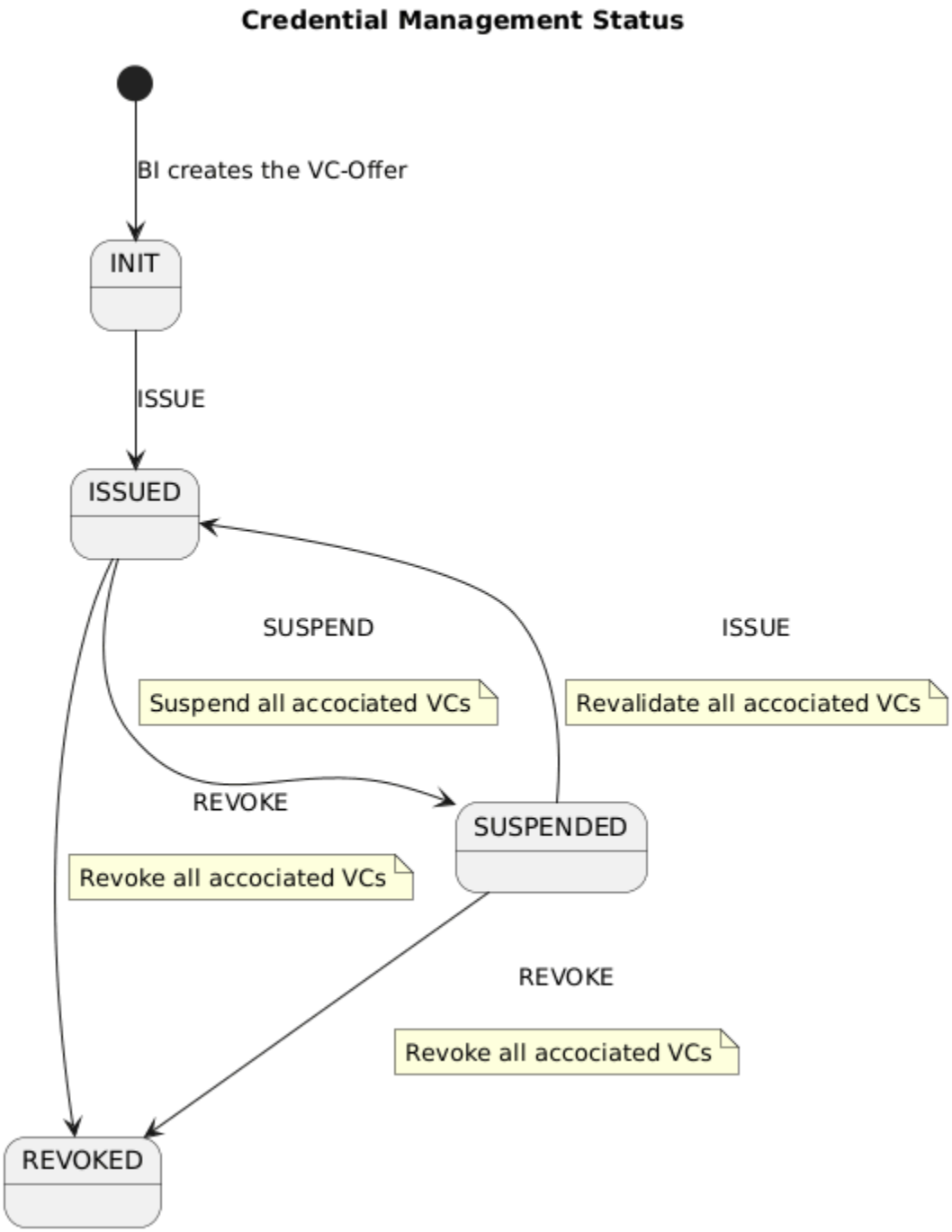
Revoked or Suspended Credentials cannot be renewed. Reason for this is the Status List handling which must be considered when issuing a credential with a status set. The Status List updates has 2 modes: Manual and Automatic. Both would create issues when allowing renewal.

- With automatic updates, each renewal does trigger an update to the status list API. This allows observers to link the indexes of newly issued VCs, defeating unlinkability.
- With manual updates, the business issuer must manually trigger an update of the status list. This means that credentials which are issued while in theory being revoked or suspended, would show and be verified as valid until the business issuer triggered an update.

Due to these issues, the generic issuer will **refuse** the renewal of VCs that have a status set. The wallet will be sent an credential_request_denied credential response error when trying to renew such a credential.

17. Runtime - Issuer - State-Event-Diagramm

17.1. Credential Management Status



The state machine for the **Credential Management** represents the overall state of all associated offers and controls the **SUSPENDED** and **REVOKED** states.
The **REVOKED** state must correlate with the corresponding state in the status lists.

17.1.1.

Renewal in Suspended / Revoked

Renewal in Suspended / Revoked

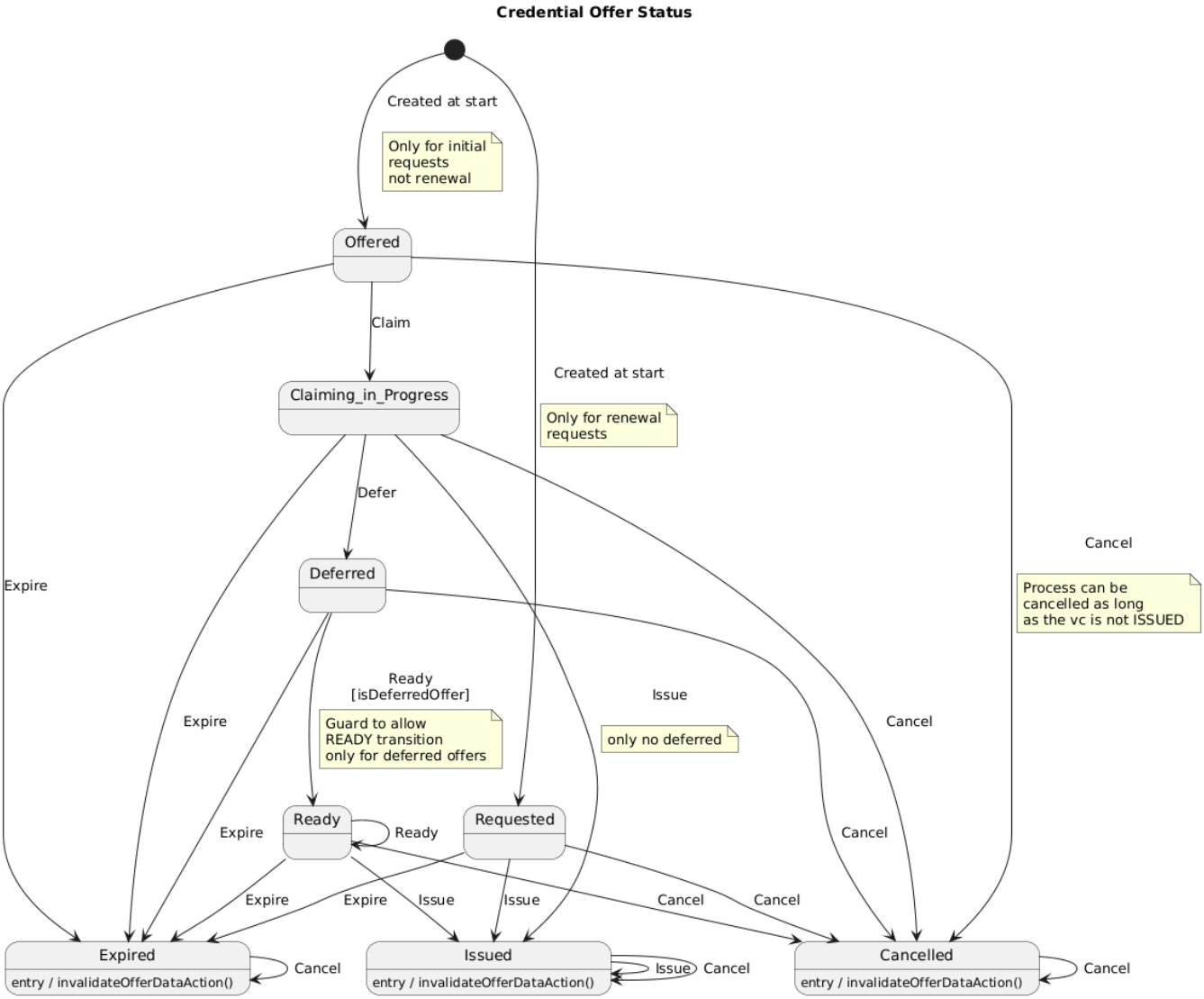
If the Management Entity is in the state **Suspended** or **Revoked**, renewal is currently not permitted and the renewal request is aborted.

The reason is that new status list entries are currently always initialized with `0`, which corresponds to **"Issued"**. Setting the status to **Suspended** or **Revoked** can take some time due to the database structure.

As a result, we cannot guarantee that newly created VCs within a batch will receive the status **Revoked** or **Suspended** before they are presented for the first time.

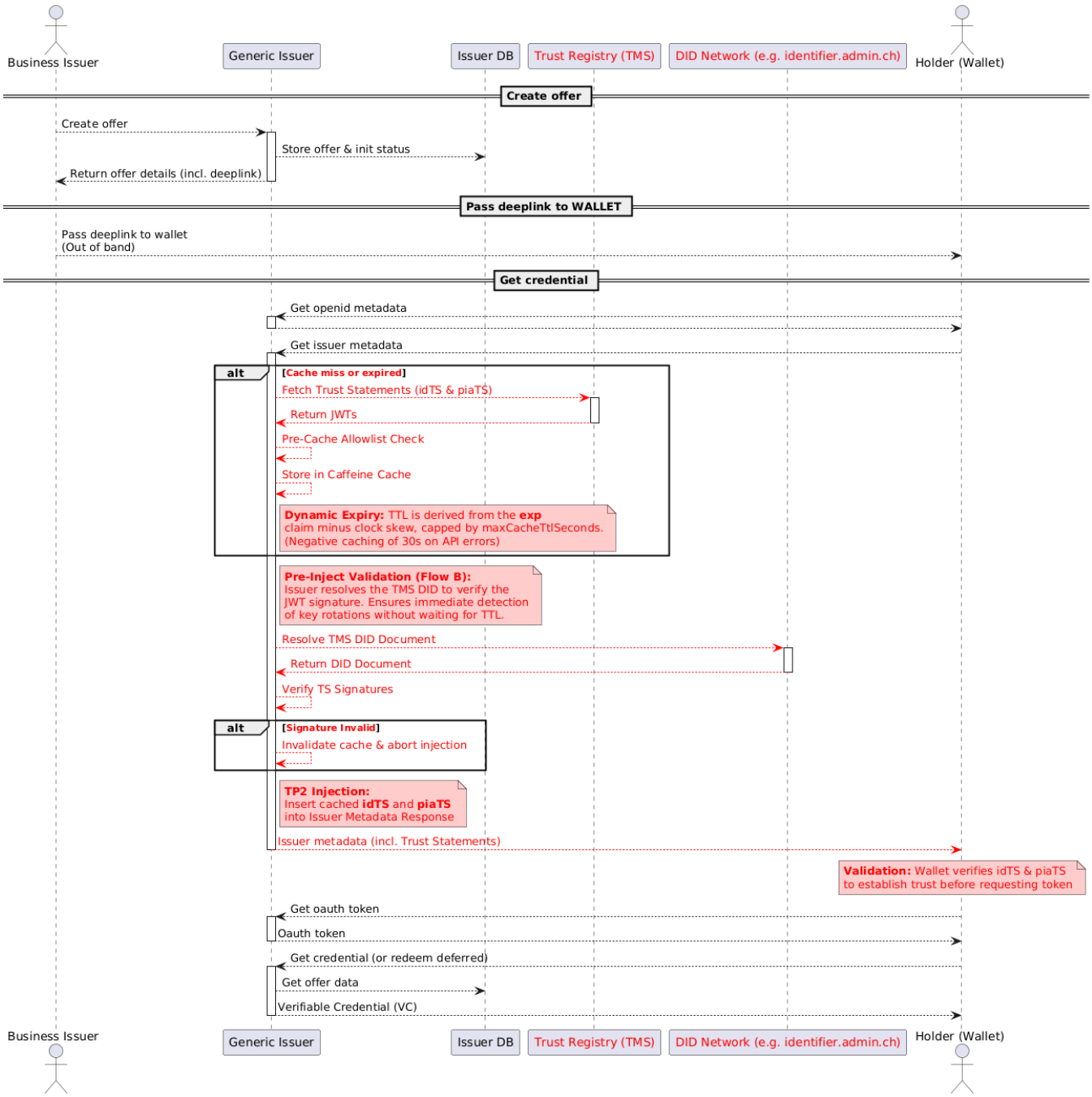
Due to this decision, **Suspended** (and **Revoked**) VCs may potentially be trackable if they are presented frequently.

17.2. Credential Offer Status



A new state called REQUESTED will be introduced.
This state indicates that a new offer has been requested via the credential renewal flow.
The credential offer remains in this state until the Business Issuer provides the necessary information and status lists, at which point the state transitions to READY.

18. Runtime - Trust Infrastructure Interactions



In the swiyu ecosystem, establishing a cryptographic trust relationship between the Holder (Wallet) and the Issuer is a mandatory prerequisite before any Verifiable Credential can be issued. According to Trust Protocol 2.0, this is achieved by supplying the Wallet with the required Trust Statements directly during the issuance flow.

To optimize performance and reduce external network dependencies for the Wallet during the interaction, the Generic Issuer implements a robust on-demand fetching, caching, and validation mechanism:

On-Demand Fetching & High-Performance Caching
Instead of static caching abstractions, the Generic Issuer utilizes a high-performance, dynamic in-memory cache (Caffeine). When the Wallet requests the Issuer Metadata for the first time, the Issuer queries the Trust Registry (TMS) API on the fly. It retrieves its own Identity Trust Statement (idTS) and, if applicable, its Protected Issuance Authorization Trust Statements (piaTS). To protect the infrastructure from retry storms during TMS outages, API failures or empty responses are negatively cached for a short duration.

Dynamic Cache Eviction matching TS Lifetime
To ensure that the Generic Issuer never serves expired Trust Statements to the Wallet, the cache implements a dynamic eviction strategy. Upon fetching a Trust Statement from the TMS, the issuer inspects the JWT payload to extract its exp (expiration time) claim. The Time-To-Live (TTL) of the cache entry is then dynamically derived from this claim, minus a clock-skew buffer to account for network latency. Additionally, the TTL can be capped by a configured maximum duration to align with public key caches. Once this lifetime is reached, the entry is automatically evicted, guaranteeing that the next metadata request will trigger a fresh fetch from the Trust Registry.

Pre-Inject Signature Validation (Flow B)
Before any cached Trust Statement is injected into a response, the Generic Issuer performs a real-time cryptographic validation. It dynamically resolves the Trust Registry's current DID Document to verify the JWT's signature. This crucial step ensures that if the Trust Registry rotates its keys or revokes a statement, the Issuer detects it immediately—even if the cache TTL has not yet expired. If the signature is invalid, the affected cache entries are actively evicted.

Dynamic Metadata Injection
When the Wallet scans the offer and initiates the OpenID4VCI flow, it fetches the Issuer's metadata (/.well-known/openid-credential-issuer). Following the successful pre-inject validation, the Generic Issuer dynamically injects the valid, locally cached idTS and piaTS into the metadata payload.

Wallet-Side Trust Validation (out of scope from Generic Issuer)
Before requesting an OAuth token or the actual credential, the Wallet extracts and evaluates the provided Trust Statements. It verifies the signatures against the Trust Registry's root anchor and validates the business rules, such as ensuring that the iss claim in the issuer metadata matches the sub claim of the piaTS. The Wallet also checks if the issuer is explicitly authorized to issue the requested credential type (e.g., protected governmental credentials). Only upon successful validation does the Wallet proceed with the credential request.

19. Runtime - Issuance with Transaction Codes

19.1. Context and Scope

19.1.1. Context

In the standard OpenID for Verifiable Credential Issuance (OID4VCI) Pre-Authorized Code Flow, a Verifiable Credential (VC) is offered via a QR code or a deep-link containing a pre-authorized code. If this link or QR code is intercepted, phished, or stolen, an unauthorized actor could collect the credential into their own wallet.

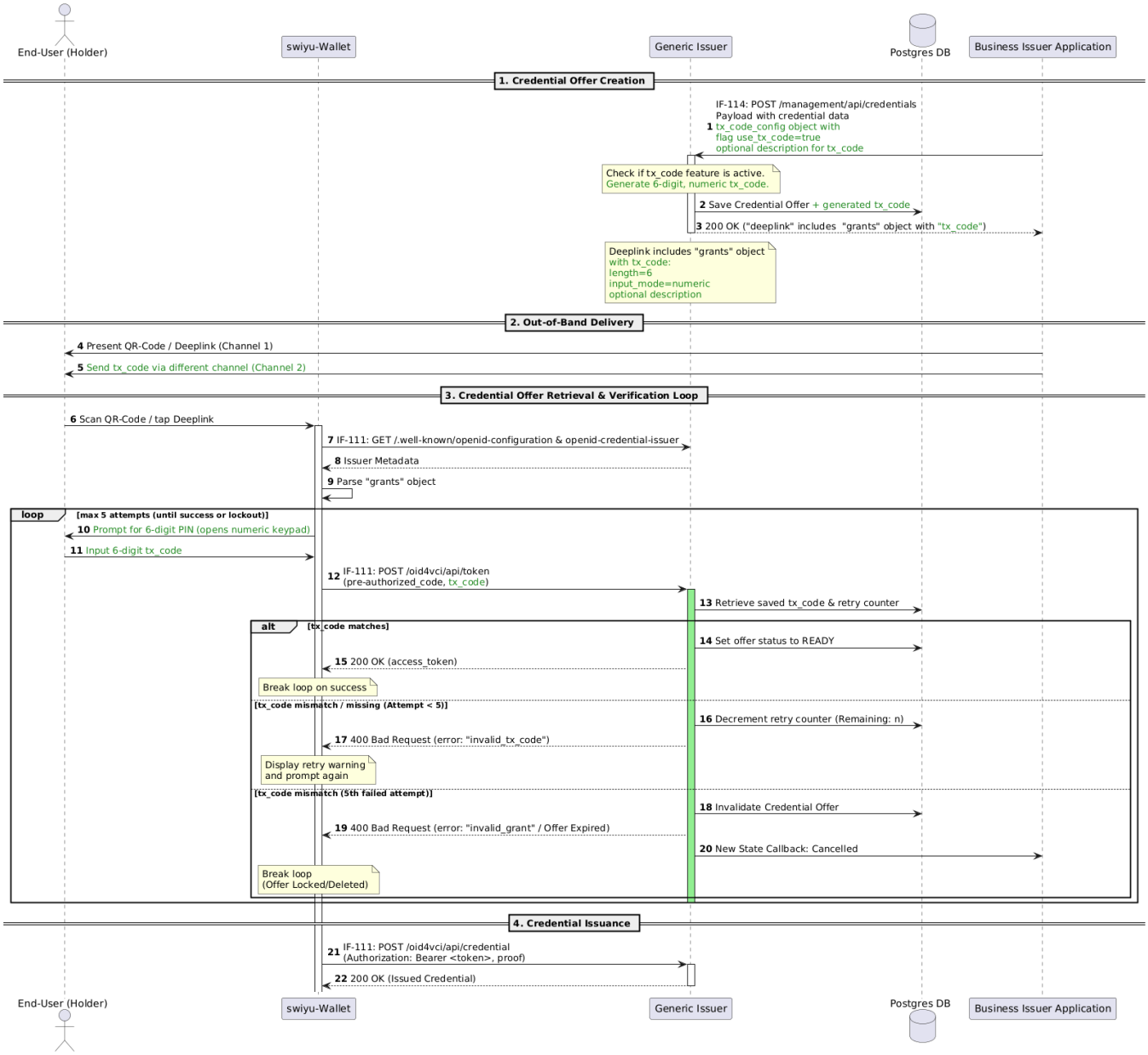
To mitigate this security risk, the Swiss e-ID ecosystem implements **OMNI-ADR-005 - Transaction Codes**. This architectural decision mandates the use of a secondary, out-of-band verification factor—the Transaction Code (`tx_code`)—for high-security credentials such as the electronic Learning Driver's License (eLFA). The `tx_code` is a short-lived, 6-digit numeric PIN generated by the Generic Issuer and transmitted to the citizen via a separate communication channel (e.g., SMS or postal letter), ensuring that only the rightful holder can successfully pull the credential.

19.1.2. Scope

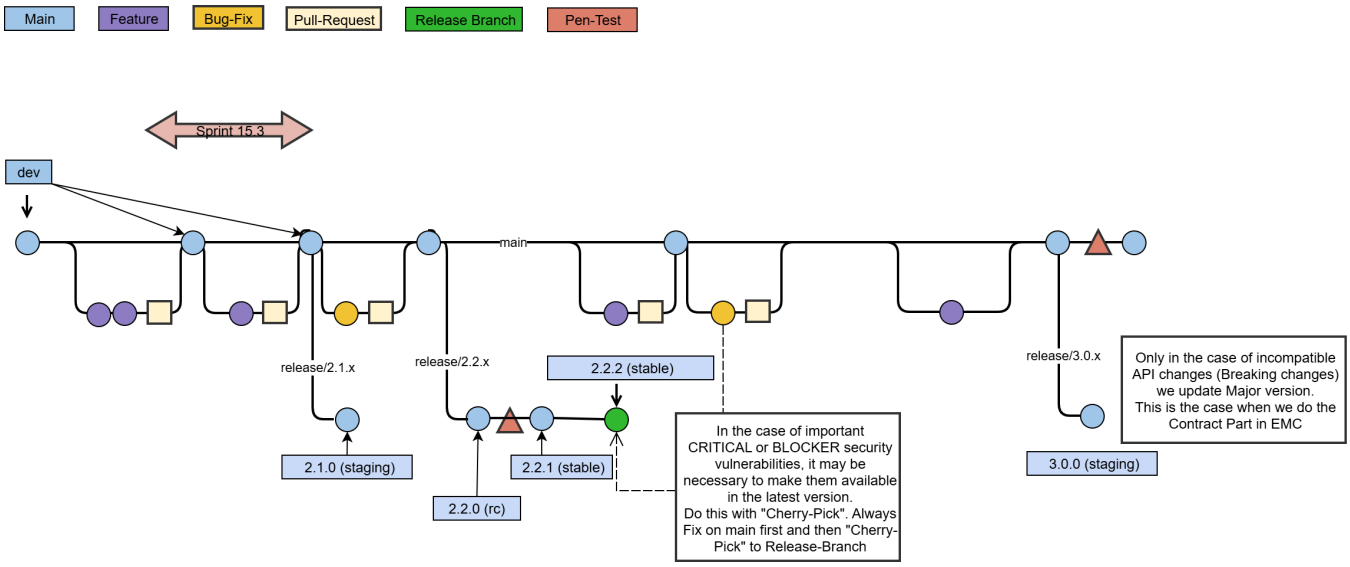
- **In-Scope:**
  - **Generic Issuer Backend:** Generation of a secure, random 6-digit numeric `tx_code` and persisting it in the database against the credential transaction.
  - **Management API (IF-114):** Extending the credential offer creation endpoint to return the generated `tx_code` to the Business Issuer application.
  - **Metadata Signalling:** Incorporating OID4VCI compliant `tx_code` metadata (`length: 6`, `input_mode: "numeric"`) within the public Credential Offer JSON to optimize Wallet UI presentation.
  - **Token Endpoint Validation (IF-111):** Verifying the `tx_code` submitted by the wallet during the token request and implementing brute-force rate-limiting (maximum of 5 attempts before the offer is permanently invalidated).
- **Out-of-Scope:**
  - The graphical user interface (GUI) layout and precise screen transitions of the PIN input screen inside the swiyu-Wallet app.
  - The actual external message delivery pipeline (SMS gateway or mailing systems used by the Business Issuer).

19.2. Runtime View

This diagram details the sequence of API interactions and data validations occurring across the Business Issuer, Generic Issuer, and swiyu-Wallet. Highlighted are the changes from the normal flow when using transaction codes.



20. Crosscutting - Branches / Releases Concept



20.1. Naming Conventions of Branches

Type	Pattern	Example
Feature-Branch	feature/<JIRA-NR>_<JIRA-SUMARY>	feature/EIDOMNI-254_Some_Description_of_bla_bla
Bug-Fix-Branch	bugfix/<JIRA-NR>_<JIRA-SUMARY>	bugfix/EIDOMNI-123_Some_Description_of_bla_bla
Release-Branch	release/<RELEASE_NUMBER>	release/2.1.x

20.2. Semantic Versioning (SemVer) Overview

Semantic Versioning (SemVer) is a versioning scheme that makes it easy to understand the impact of a software release just by looking at the version number. A SemVer version number follows the format:

```
MAJOR.MINOR.PATCH
```

- **MAJOR version (X.y.z)**
  Incremented when we **Contract** the system by removing, changing, or breaking existing features.
  - Example: Removing a deprecated endpoint, changing response formats in a non-compatible way.
- **MINOR version (x.Y.z)**
  Incremented when we **Extend** the system with new, backward-compatible functionality.
  - Example: Adding a new endpoint, introducing an optional field, or extending valid inputs.
- **PATCH version (x.y.Z)**
  Incremented when we **Maintain** the system with backward-compatible security fixes on Release Branch.
  - Example: Security Bug fixes or important performance optimizations where are also needed on last Release
  - **Security fixes for the release branch must always originate from the main branch.**

This system provides a predictable contract between maintainers and users: if you upgrade within the same MAJOR version, your existing integrations should continue to work. (Step Expand and Migrate in EMC Pattern)

20.2.1. Security Fix Process on release branch

Policy: Upstream First ("Main First")

As a non-negotiable rule, all bug fixes and security patches must first be developed and integrated on the main branch.
Exception: If a fix must necessarily be developed directly on a release branch due to strongly diverging code bases, a mandatory "forward-port" ticket for the main branch must simultaneously be included in the current sprint.

1. Resolve the bug or security fix on the **main** branch.
2. **Cherry-pick** the commit onto the relevant release branch.

```
git checkout release/2.1.x
git cherry-pick <commit-hash>
```

3. Run tests and verify the fix on the release branch.
4. Publish a new release with a **PATCH version bump** (e.g., `2.1.0` → `2.1.1`).
5. Document the fix in the release notes.

20.3. Releases

Docker images for this project follow a formalized environment-based tagging approach:

Tag	Meaning	Description
dev	Development Build	Latest commit from the development branch. Automatically generated on every push to `main`.
staging	Integration Test Build	Set at the end of a sprint or after completion of a feature for integration testing.
rc	Release Candidate	Frozen state prior to release and penetration testing.
stable	Verified Production	Released after successful QA and penetration testing.

These tags are assigned automatically or manually as part of the CI/CD workflow. This ensures that environments can reliably reference images by their lifecycle stage (e.g., `swiyu-issuer-service:staging`) without requiring manual version management.

20.3.1. Promotion Workflow

The image promotion process follows these steps:

```
[Commit  dev]
        build & push :dev
[Feature completed / Sprint end]
        promote  :staging
[Release candidate created]
        promote  :rc
[QA & penetration test passed]
        promote  :stable
```

20.3.2. When to create a official release

Before any offinial release can be approved, a **penetration test must be conducted**.
A release may only be made if the penetration test has been successfully completed and accepted.

- **When penetration tests are required**
  - 01. Pentest - Policy Process according to 1.5. Pentesting Decision
  - Typically every few months
  - When introducing new features
  - When making changes or adjustments to the API
- **After the tested release branch**
  On a release branch that has already passed a penetration test, additional **non-pen-test-critical security fix patches** may be integrated.
  These can be included in subsequent releases **without requiring another penetration test**.